

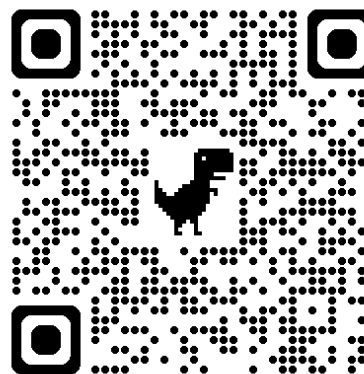


自然語言處理與應用

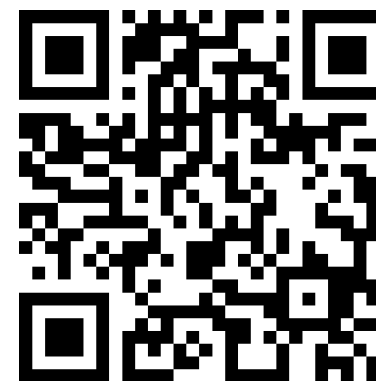
Natural Language Processing and Applications

Transformer

Instructor: 林英嘉 (Ying-Jia Lin)
2026/03/23



GitHub



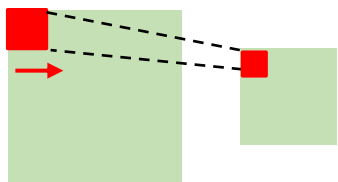
Slido
NLP0323
([Link](#))

Outline

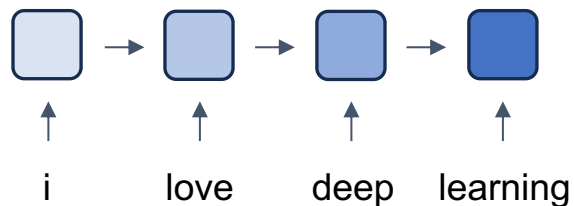
- Recaps [10 min]
- Transformers [90 min]
- Homework 2 [30 min]
- Quiz [15 min]

深度學習常見模型架構

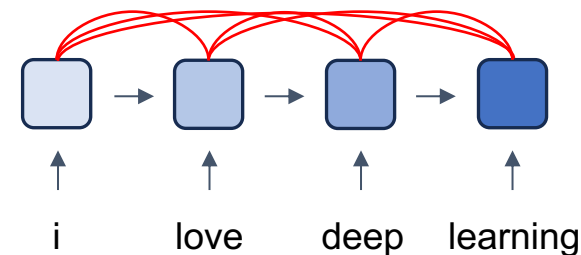
CNN (著重局部資訊)



RNN (著重記憶力)

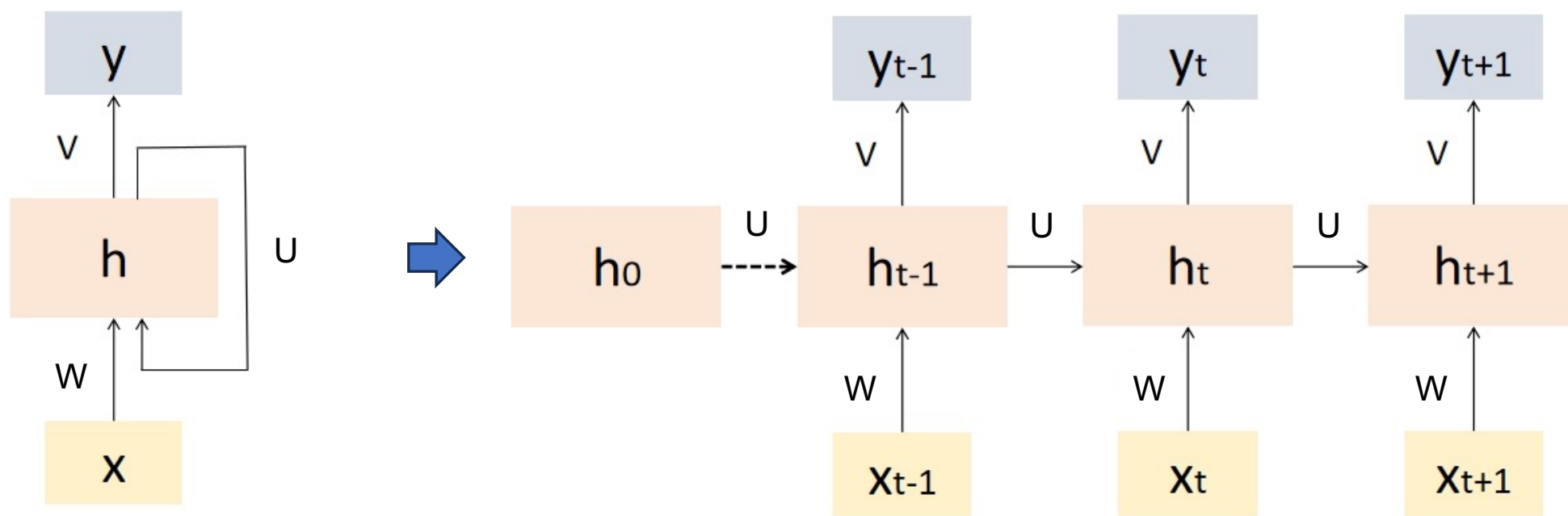


Transformer (著重全局資訊與記憶力)



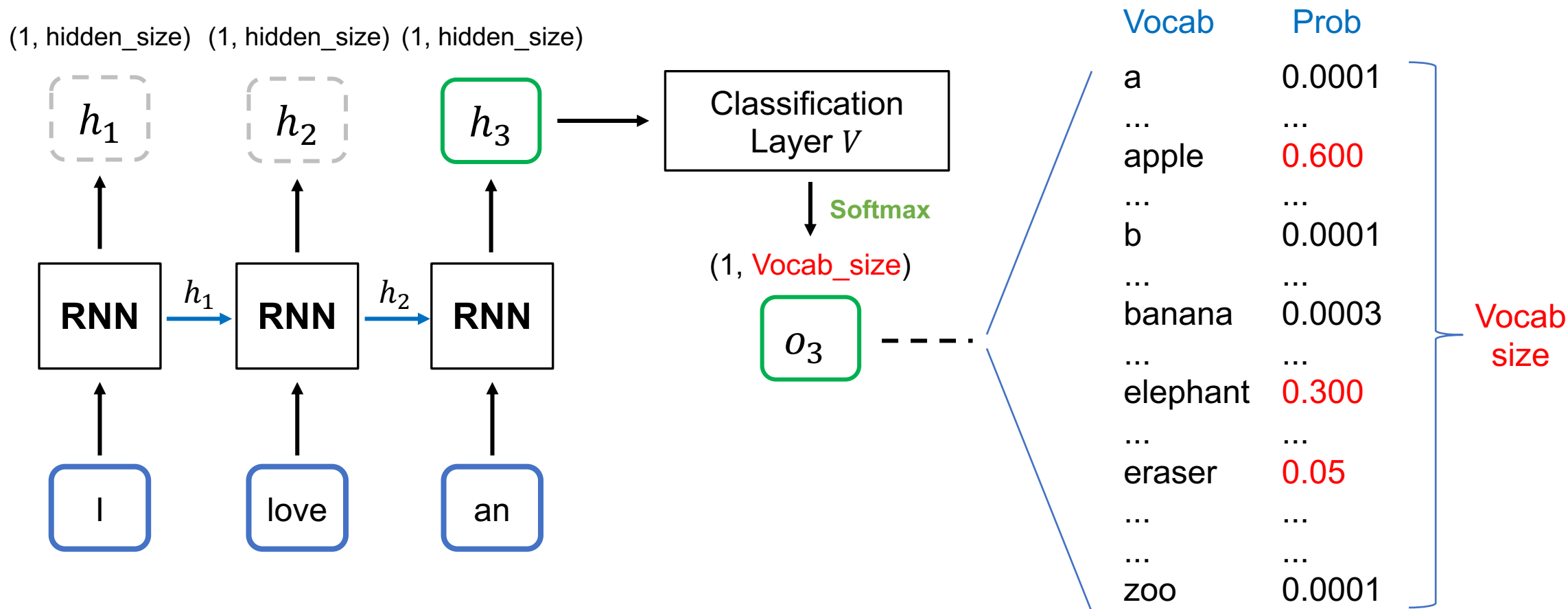
[Recap] Recurrent Neural Networks (RNN)

- Recurrent Neural Networks (RNNs) are a type of artificial neural network designed to handle sequential data by **capturing temporal dependencies**.
 - RNN 的“遞迴”是指它在每一個時間步中重複使用相同的參數（同一組權重），並把先前的隱藏狀態傳到下一個時間步

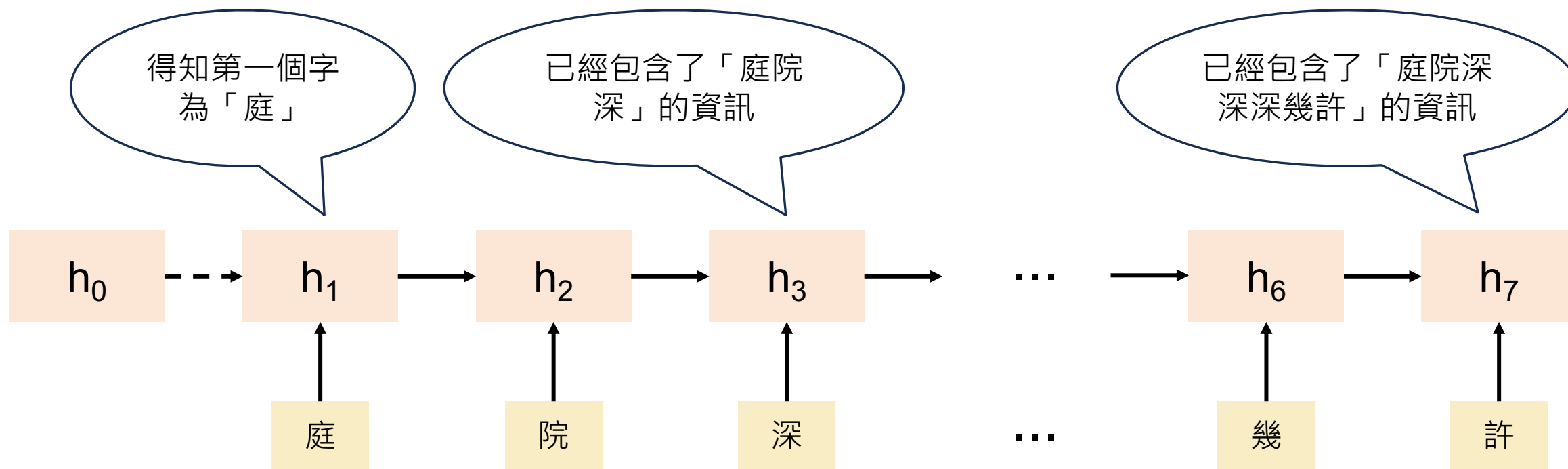


[Recap] Text Generation with RNN

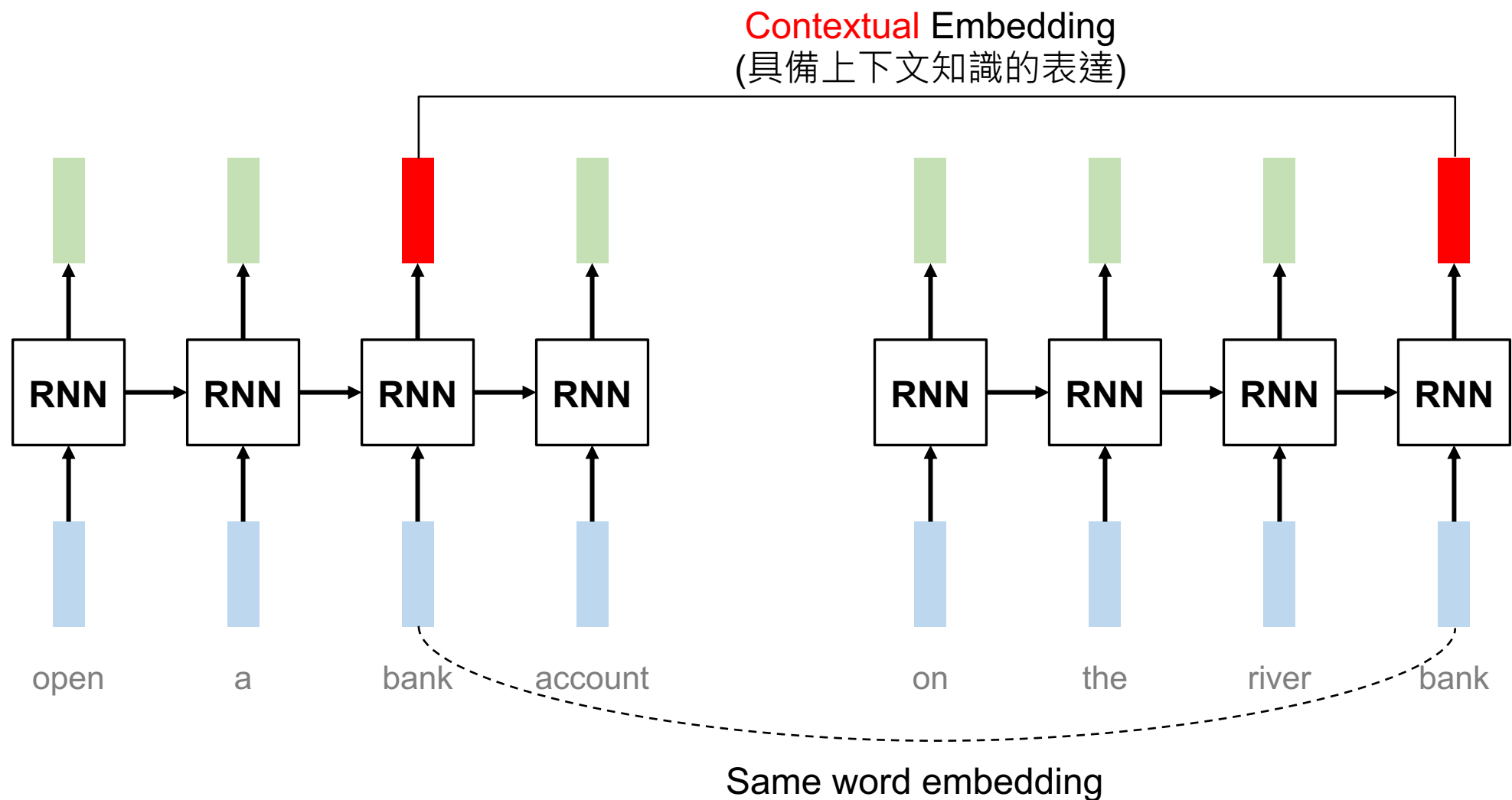
*本頁的RNN不包含輸出層 V



[Recap] RNN 的記憶力意義



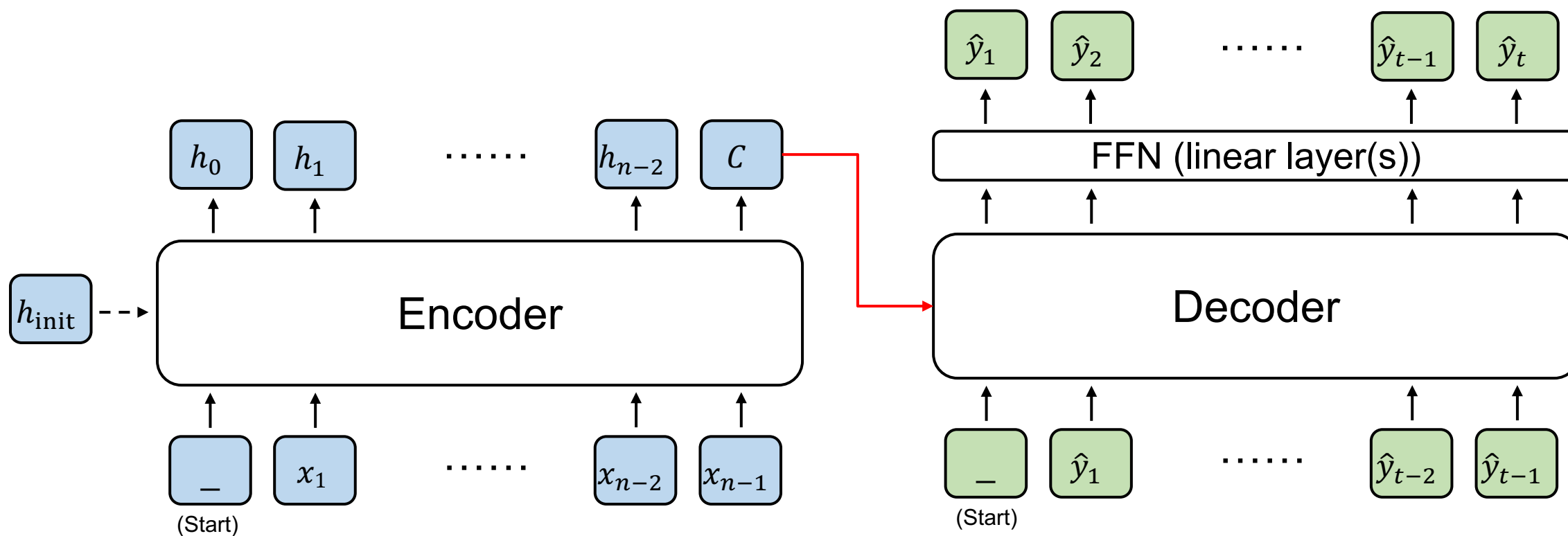
為什麼我們需要序列模型？



Encoder and Decoder

輸出是 hidden states

輸出是 sequence (文字)



Input sequence

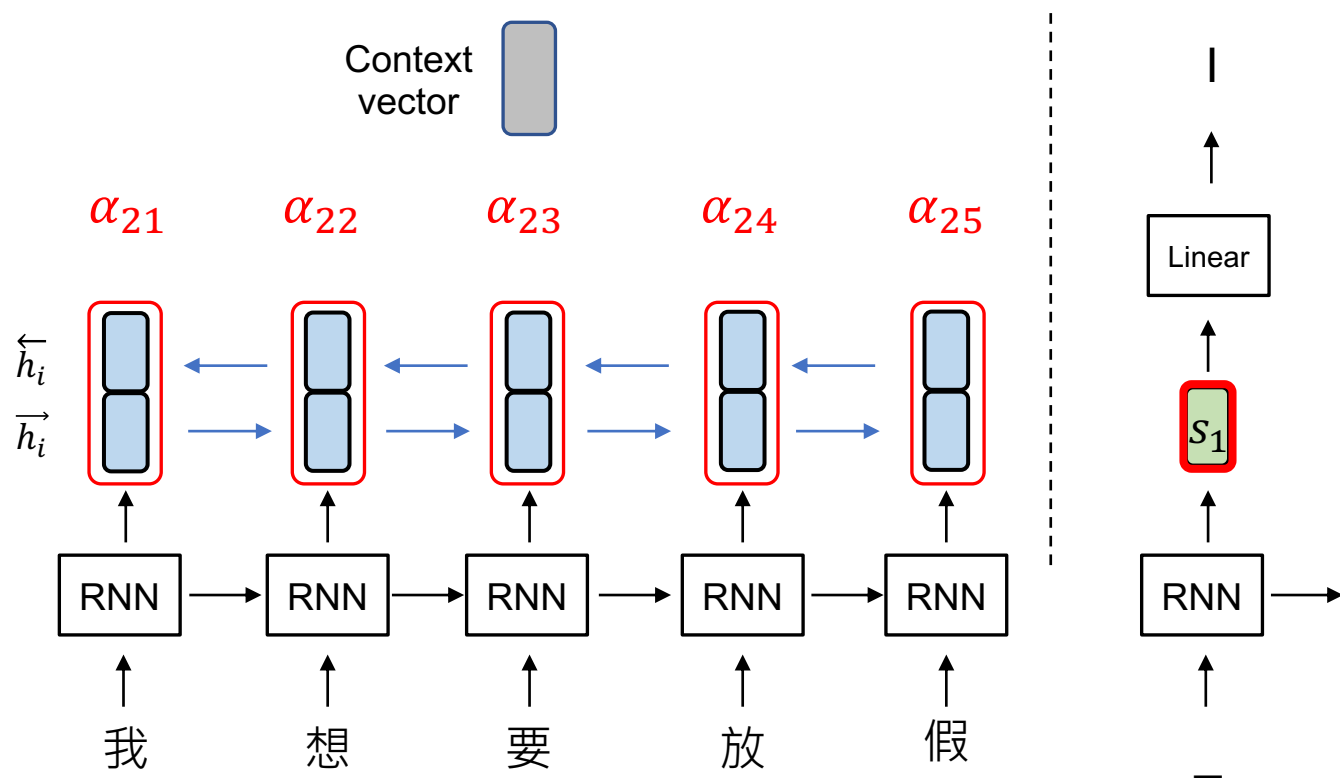
(Encoder 的輸出有很多種
被Decoder運用的方式)

機器翻譯 (with Attention, t = 2)

$$\vec{h}_i = [\vec{h}_i; \overleftarrow{h}_i]$$

$$c_2 = \sum_{i=1}^n \alpha_{2i} h_t$$

Context
vector



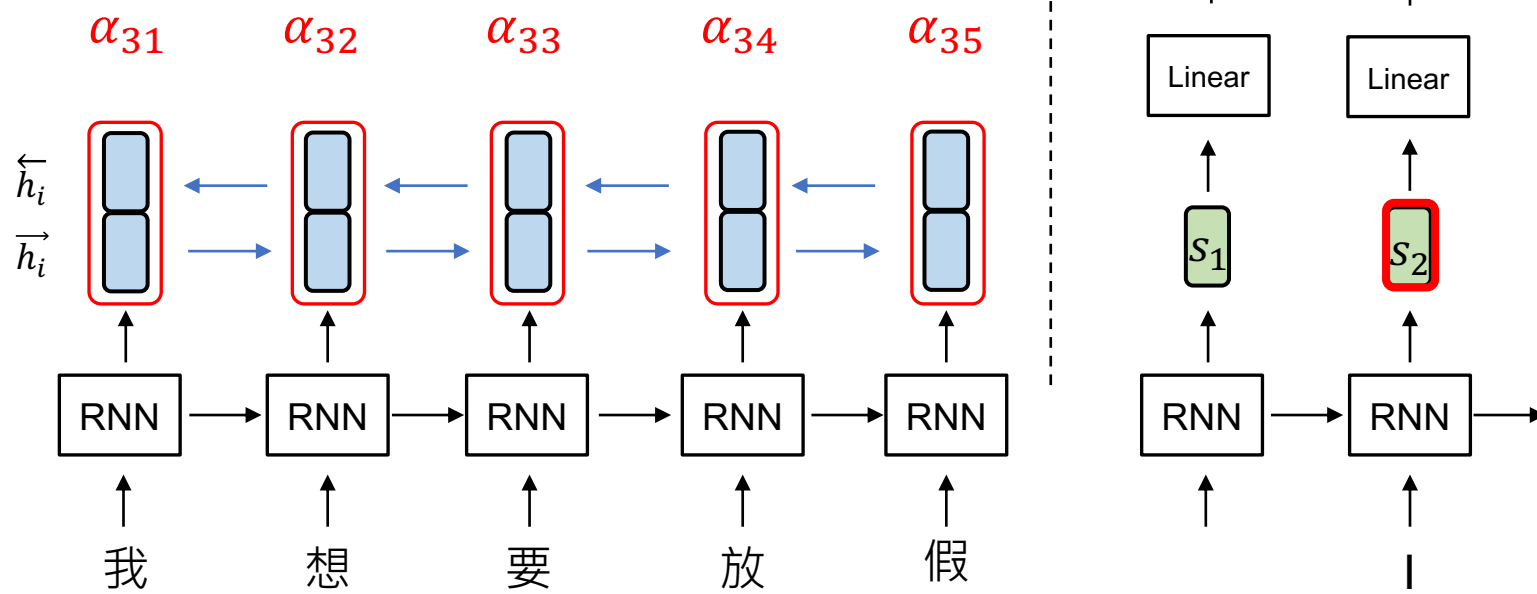
i : encoder timestep
 t : decoder timestep

機器翻譯 (with Attention, t = 3)

$$\vec{h}_i = [\vec{h}_i; \overleftarrow{h}_i]$$

$$c_3 = \sum_{i=1}^n \alpha_{3i} h_t$$

Context
vector



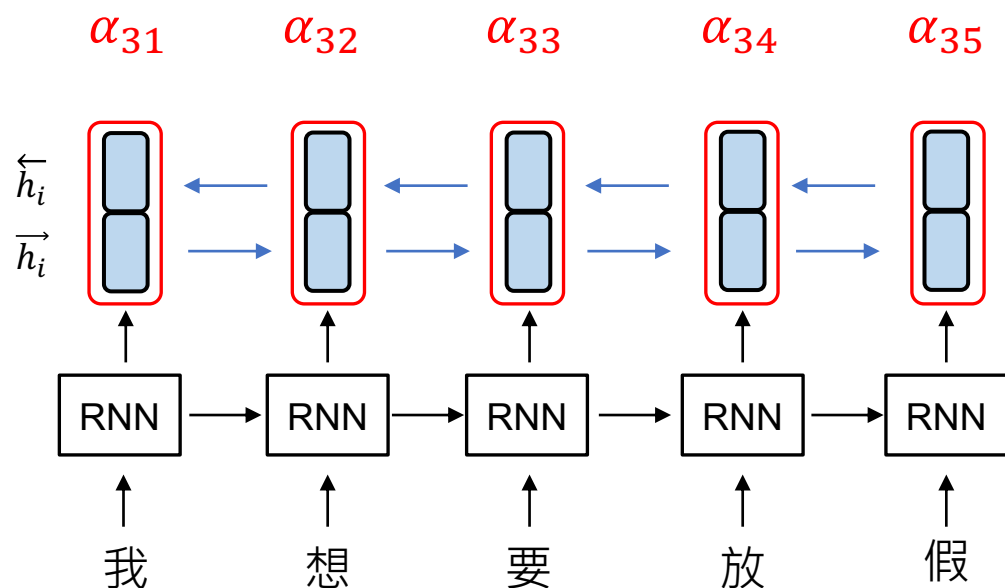
i : encoder timestep
 t : decoder timestep

Additive Attention (算出 α_{ti})

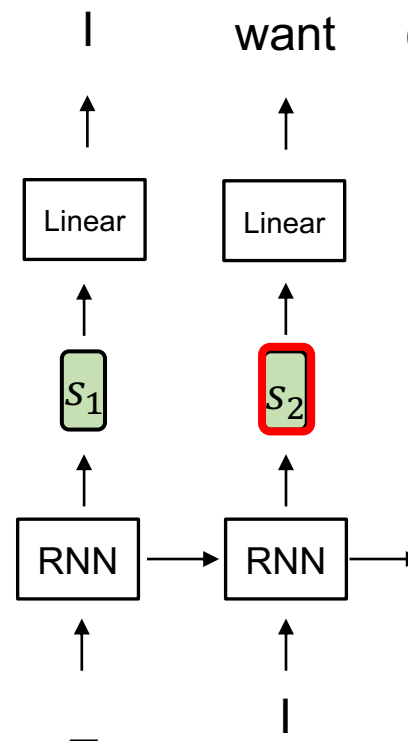
$$\vec{h}_i = [\vec{h}_i; \overleftarrow{h}_i]$$

$$c_3 = \sum_{i=1}^n \alpha_{3i} h_t$$

Context
vector



i : encoder timestep
 t : decoder timestep



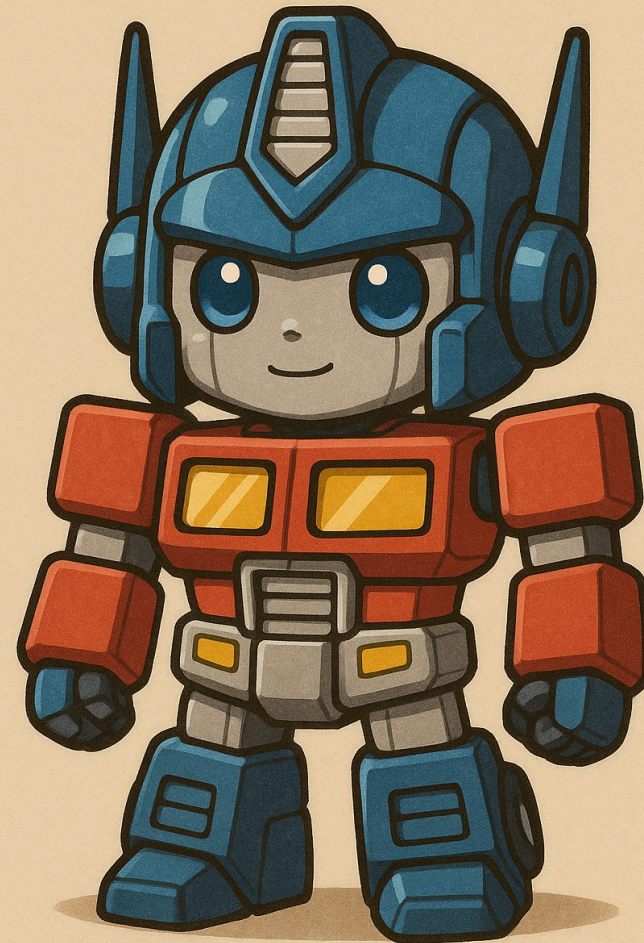
Given $i \in \{1, \dots, n\}$,

$$\alpha_{ti} = \text{softmax}(v \cdot \tanh(W s_{t-1} + U h_i))$$

Additive
Attention

轉為1個
數字

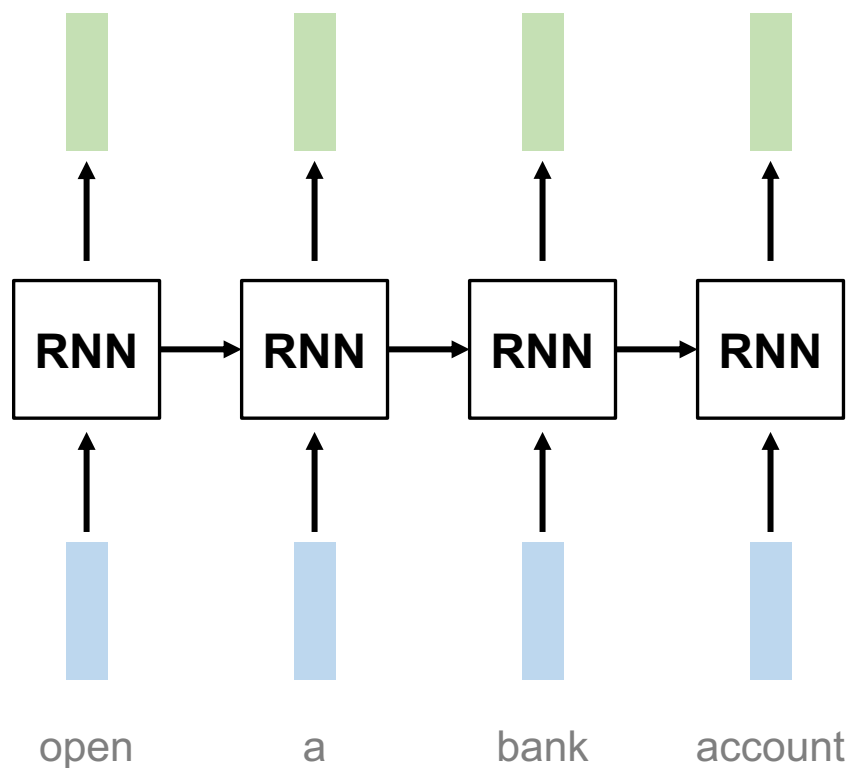
Transformer



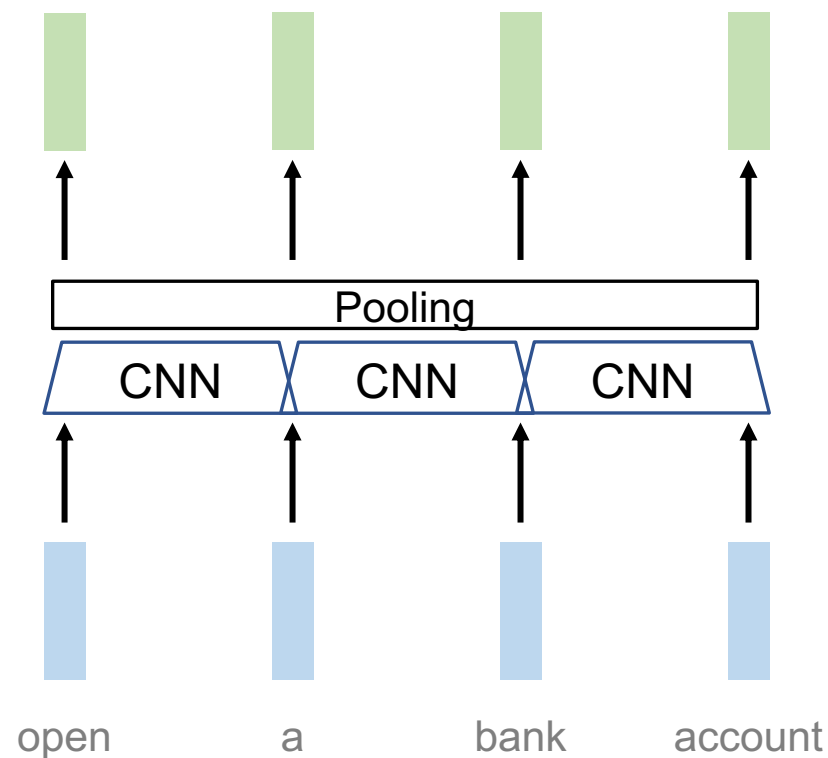
TRANSFORMER

RNN 與 CNN 作為序列模型的問題

問題：RNN 難以平行化

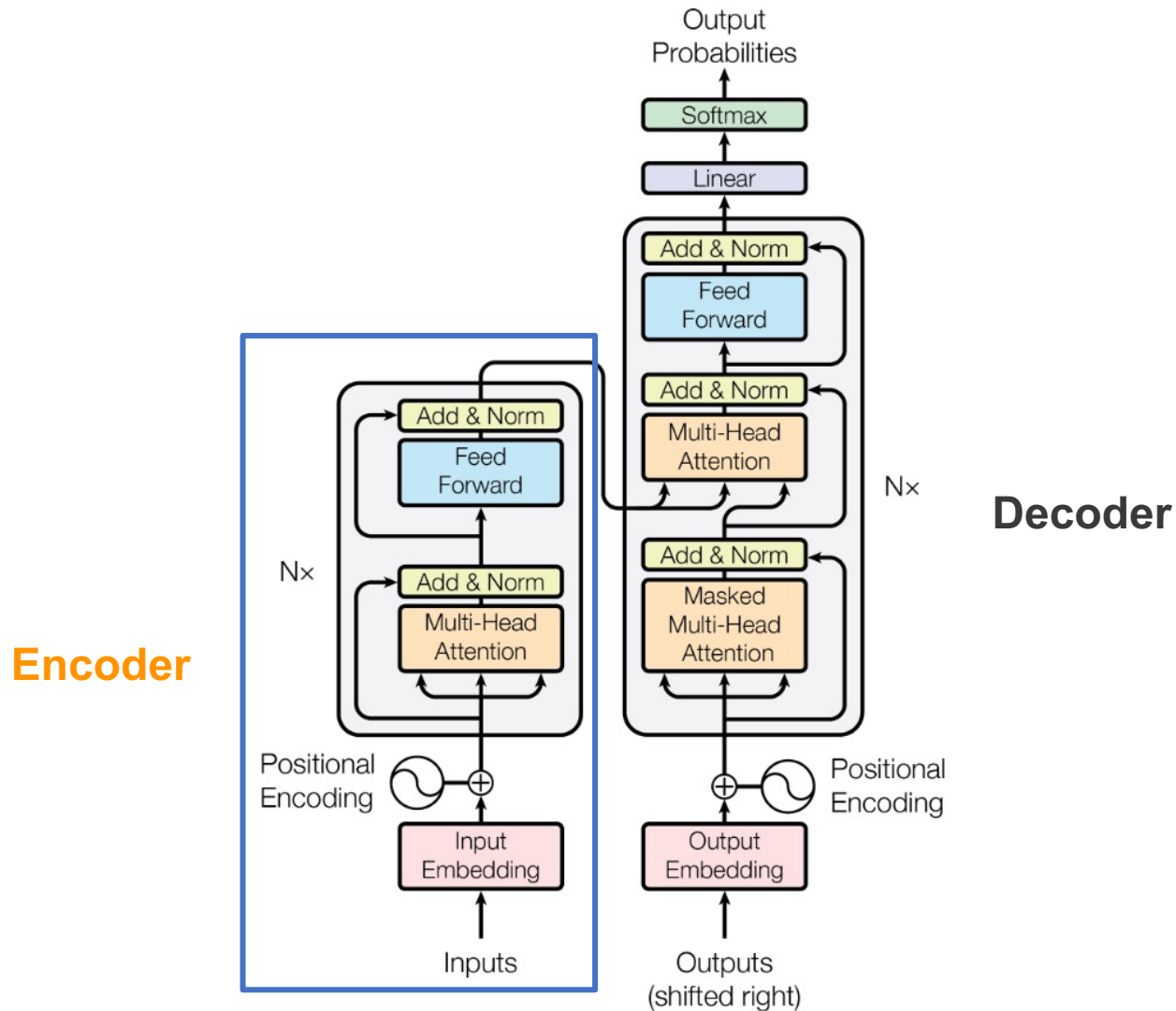


問題：CNN 著重局部資訊



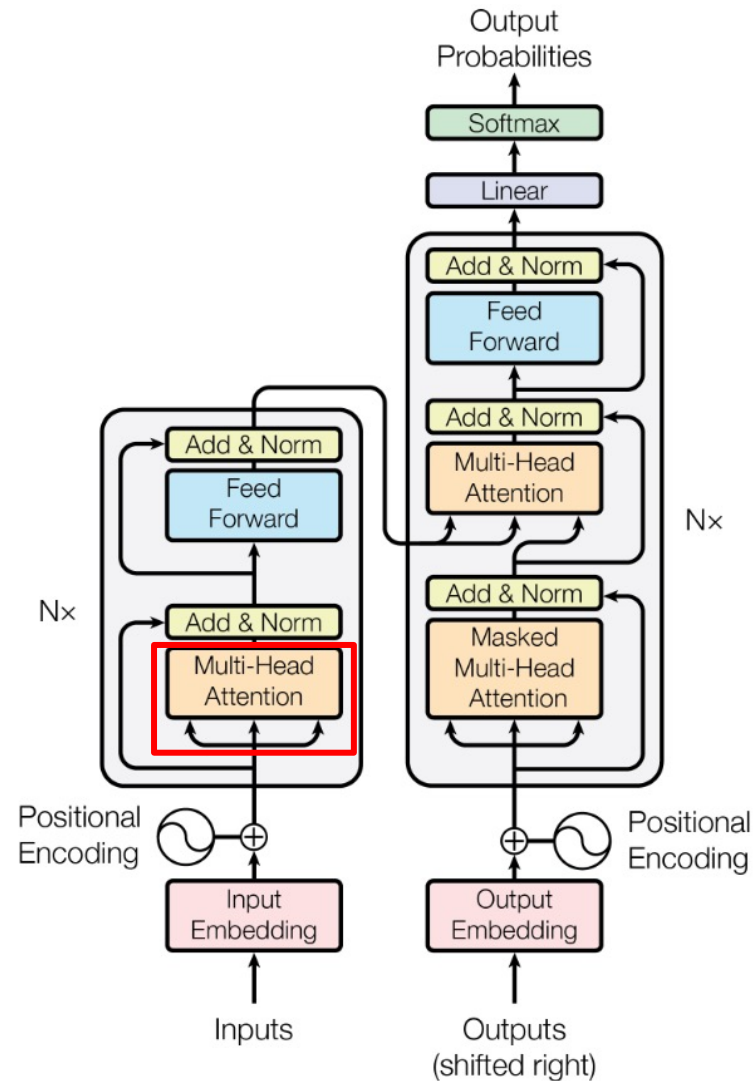
Overview of Transformer

Vaswani, Ashish, et al. "Attention is all you need." NeurIPS (2017).



Outline of Transformer

Vaswani, Ashish, et al. "Attention is all you need." NeurIPS (2017).

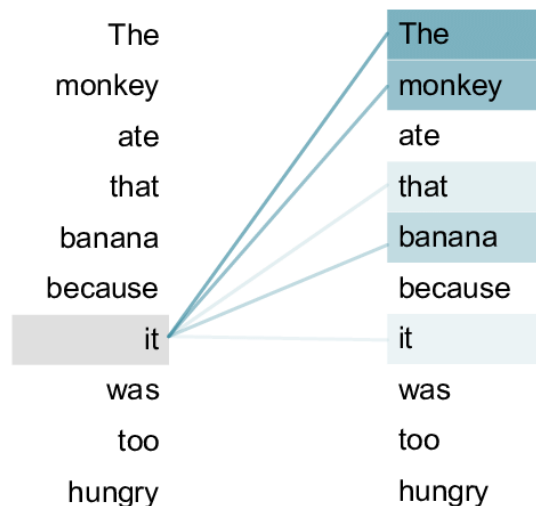


Transformer

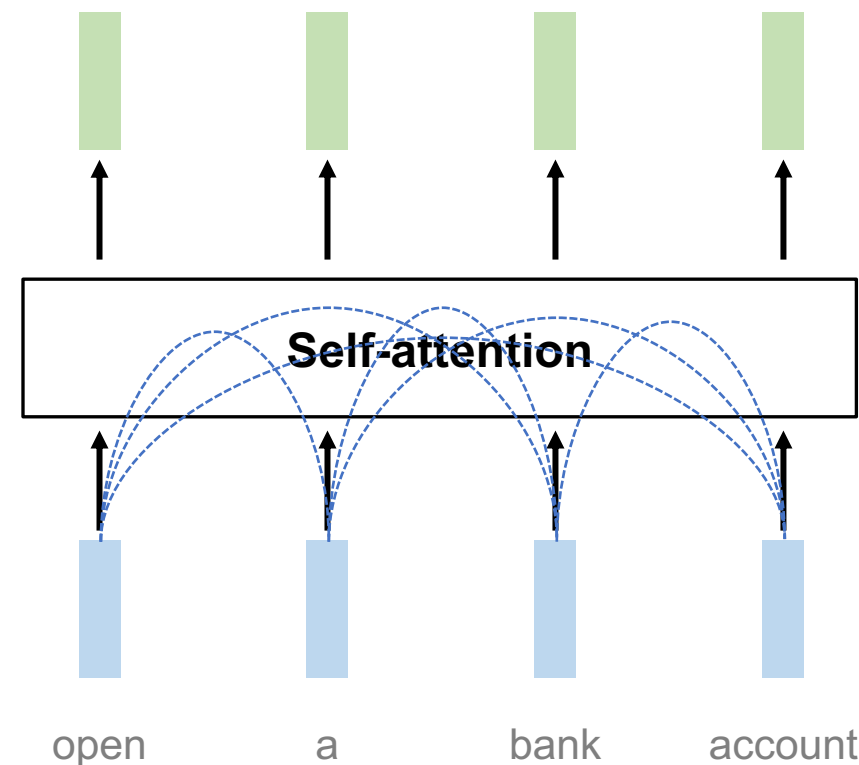
Source: [An example of the self-attention mechanism following long-distance... | Download Scientific Diagram \(researchgate.net\)](#)

- Transformer 來自論文：Attention Is All You Need (Vaswani et al., 2017)
- Transformer 內部的主要機制為 Self-attention

Self-attention: 句子內的每個 token 自己
跟自己做 attention

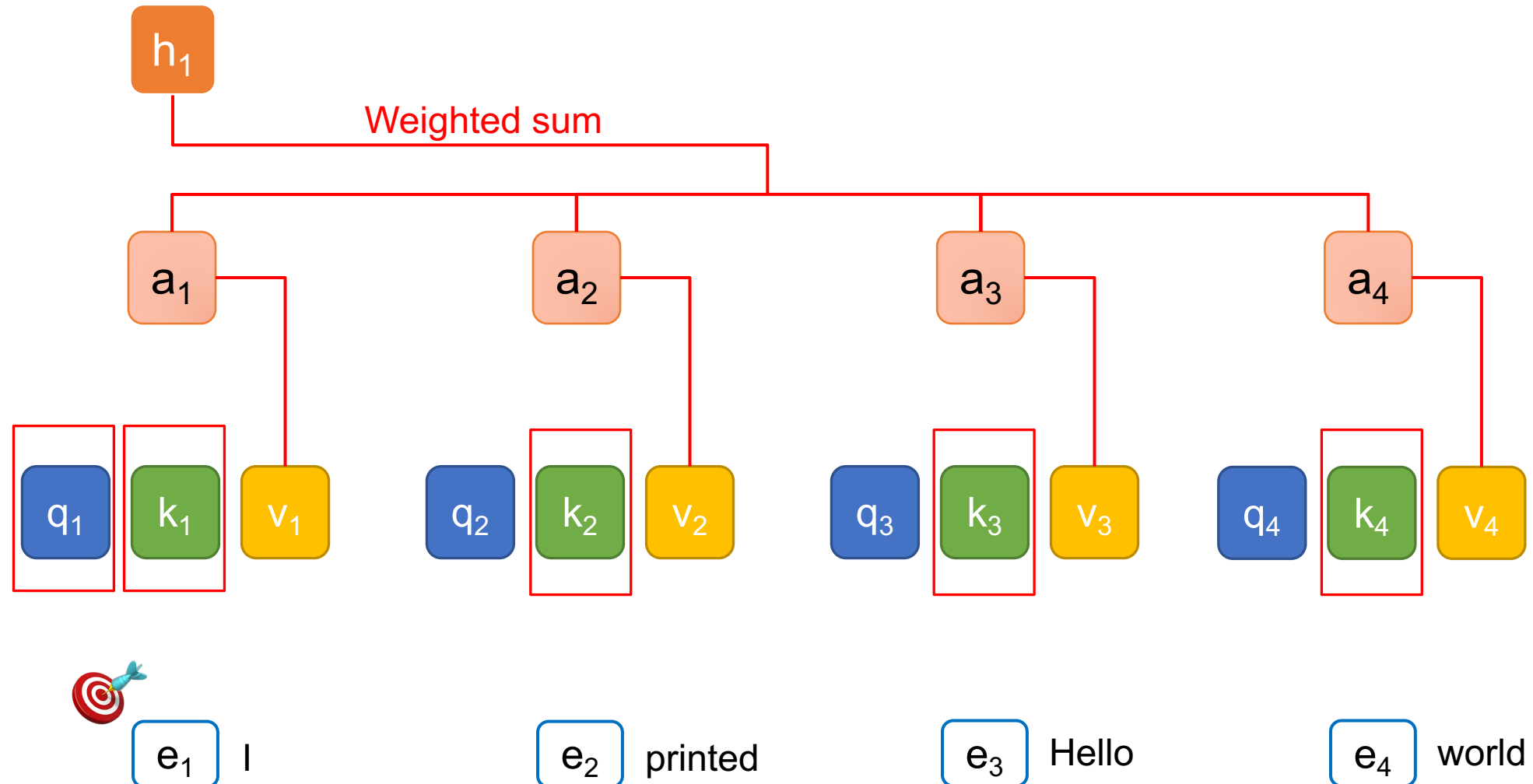


attention 進行的過程需要主動與被動



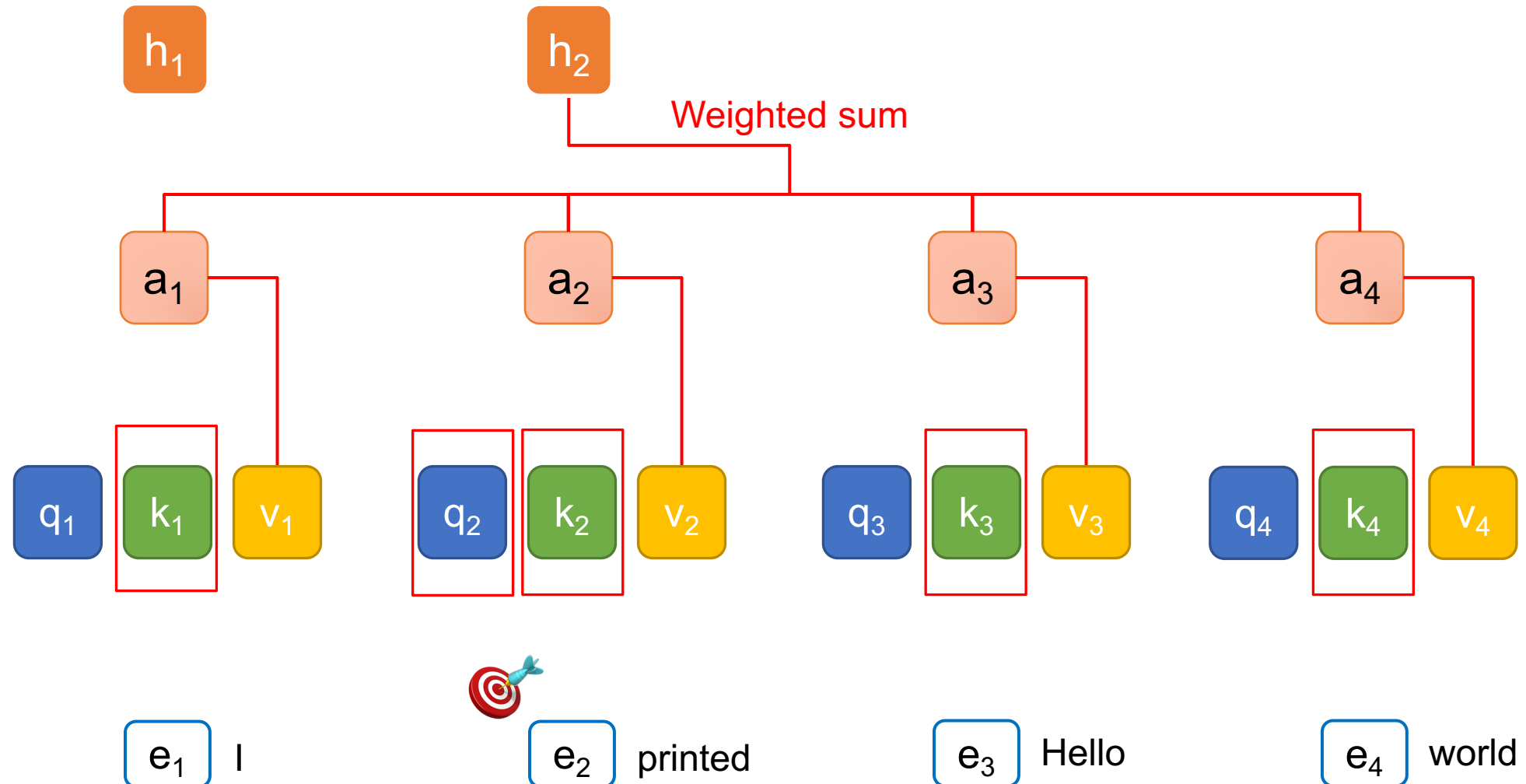
Query (Q), Key (K), and Value (V)

$t = 1$



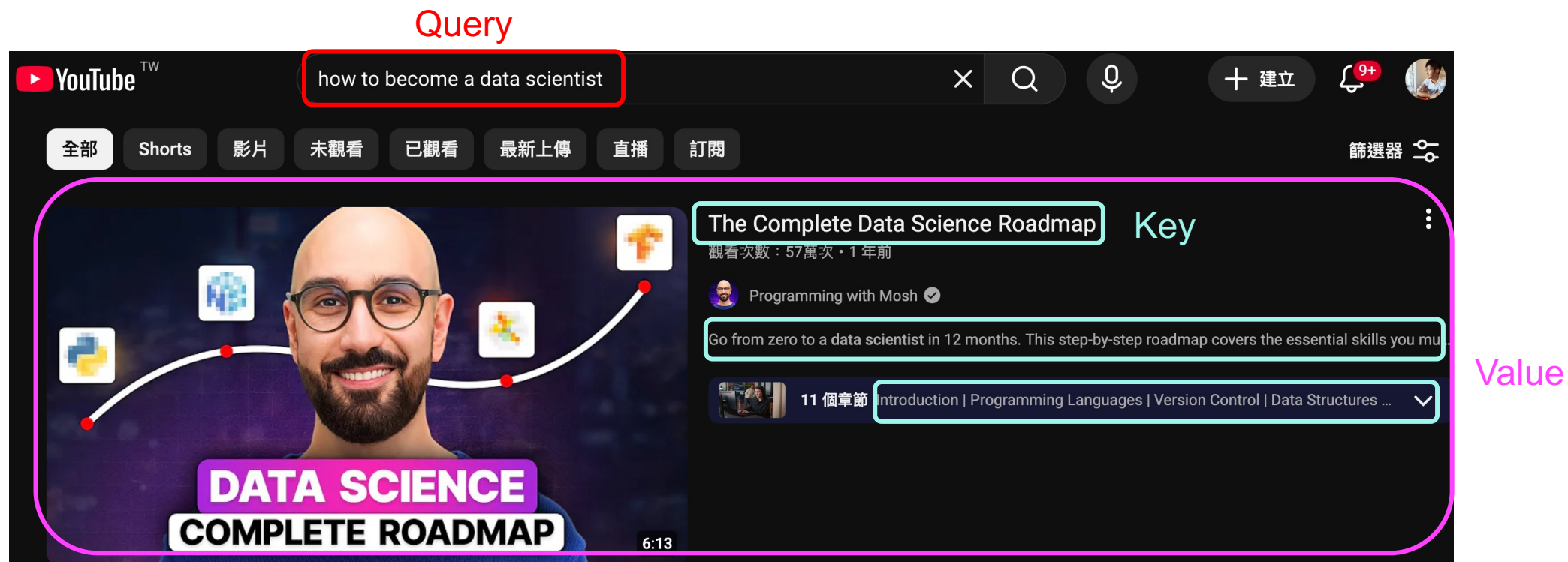
Query (Q), Key (K), and Value (V)

$t = 2$



QKV 的比喻

Vaswani, Ashish, et al. "Attention is all you need." NeurIPS (2017).



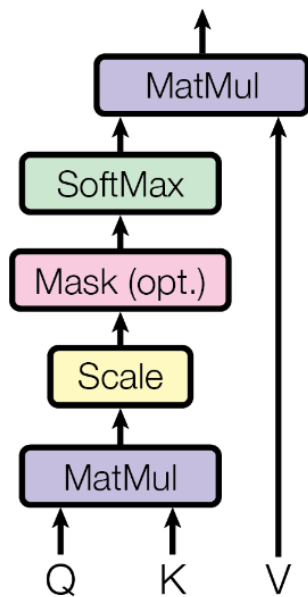
- Query: (主動) 查詢的關鍵字；Key: (被動) 查詢的對象
- Value: 值，根據關鍵字與對象匹配程度被加權整合的資訊

為什麼 Transformers 可以平行化？

d_k : queries 和 keys 的維度大小

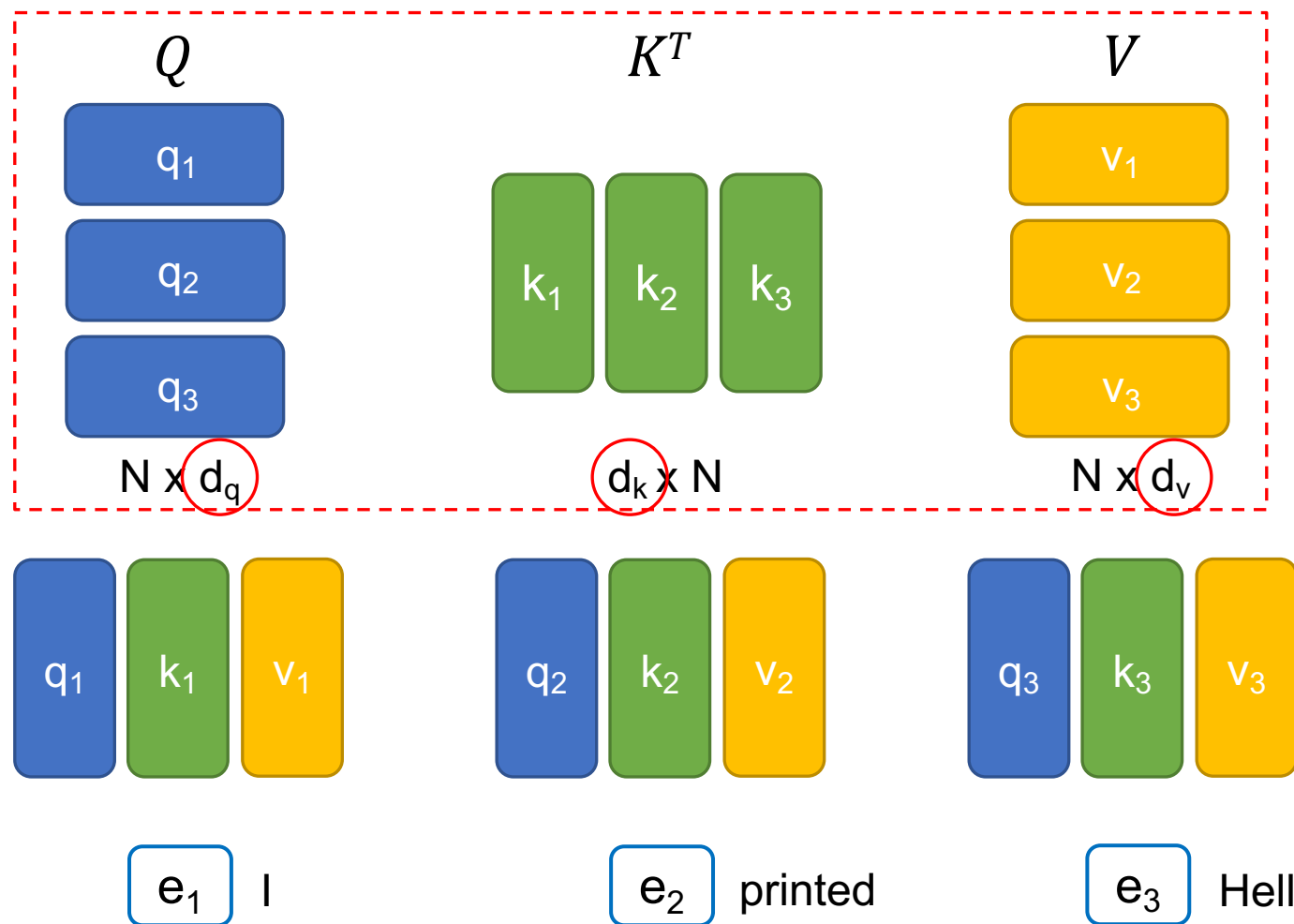
通常設置 $d_q = d_k = d_v$

Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

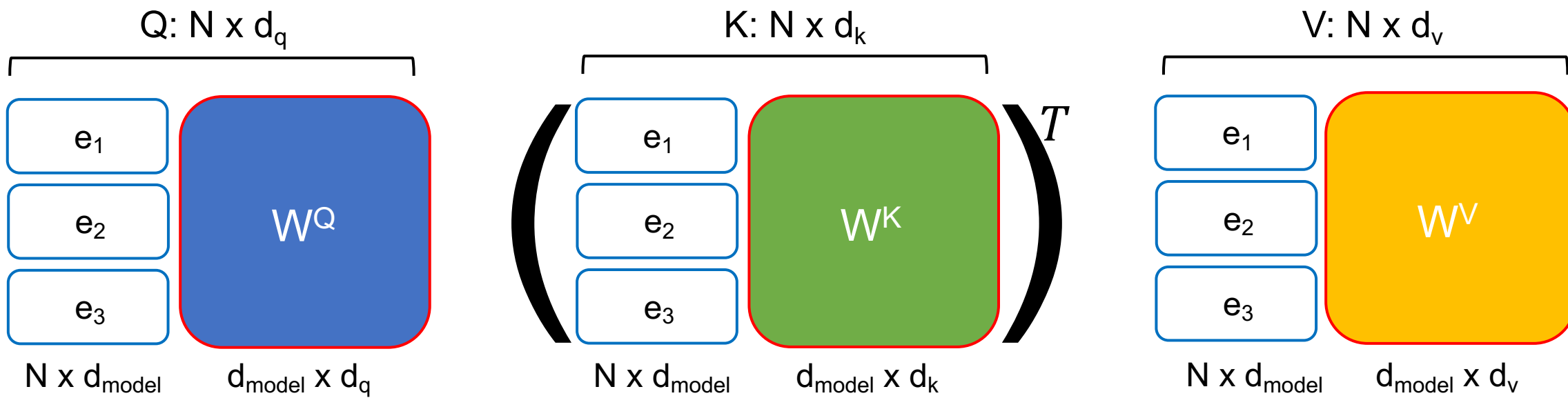
(自注意力機制可以利用矩陣乘積來進行平行化計算)



Transformers 的權重值 (1/2)

可以被訓練的參數

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



d_{model} :
Embedding
的維度大小

e_1 e_1 e_1

e_2 e_2 e_2

e_3 e_3 e_3

I

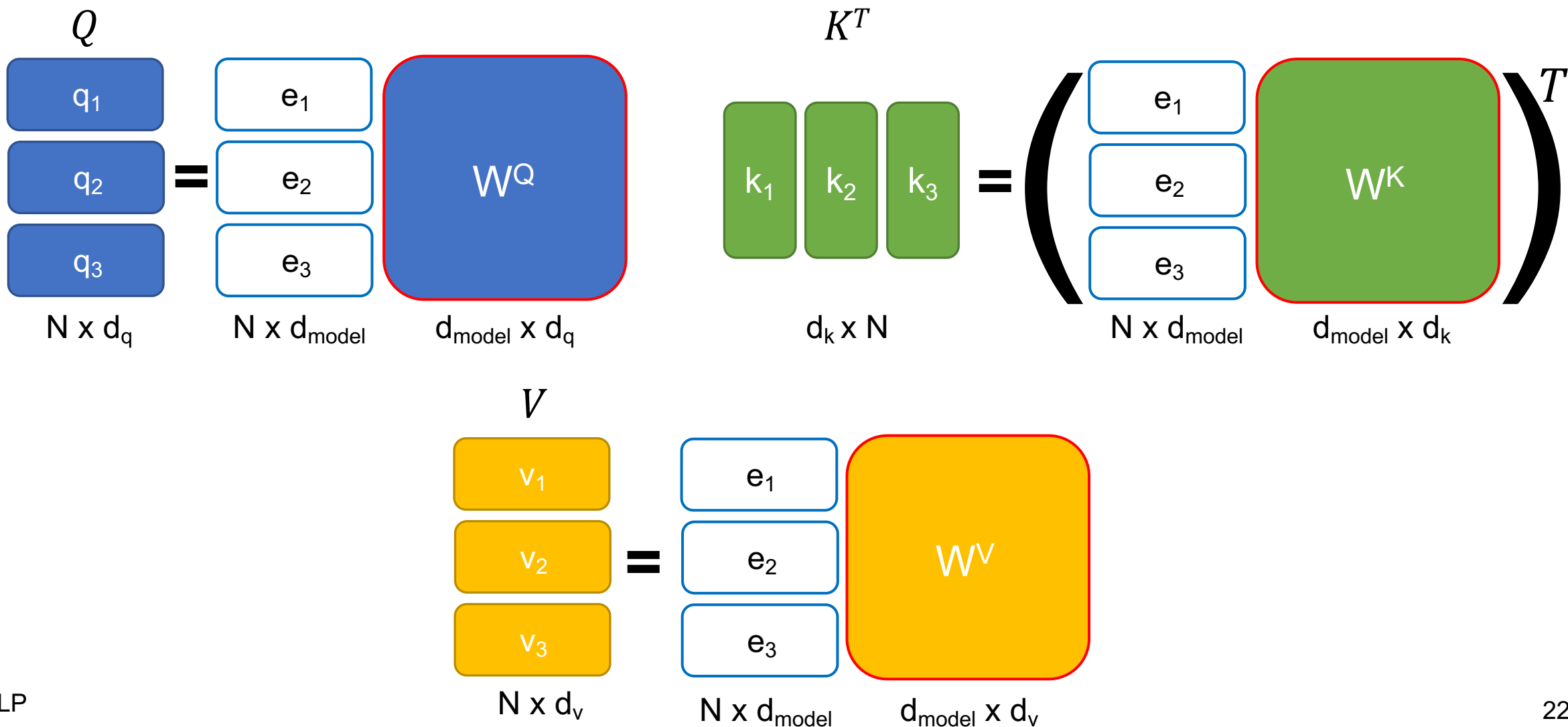
printed

Hello

可以被訓練的參數

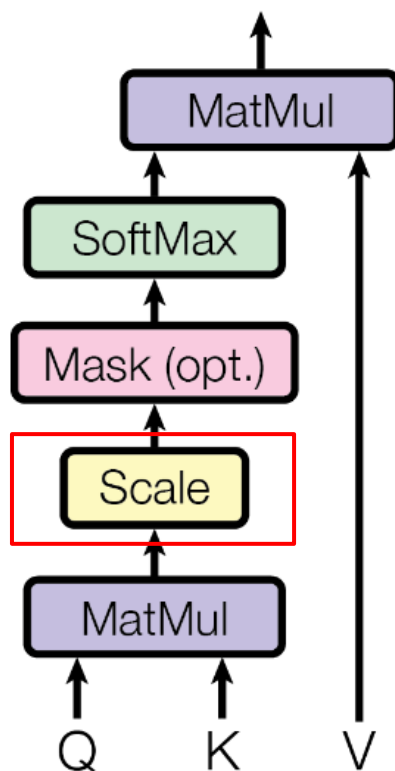
Transformers 的權重值 (2/2)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Self-attention 的公式與縮放項

Scaled Dot-Product Attention

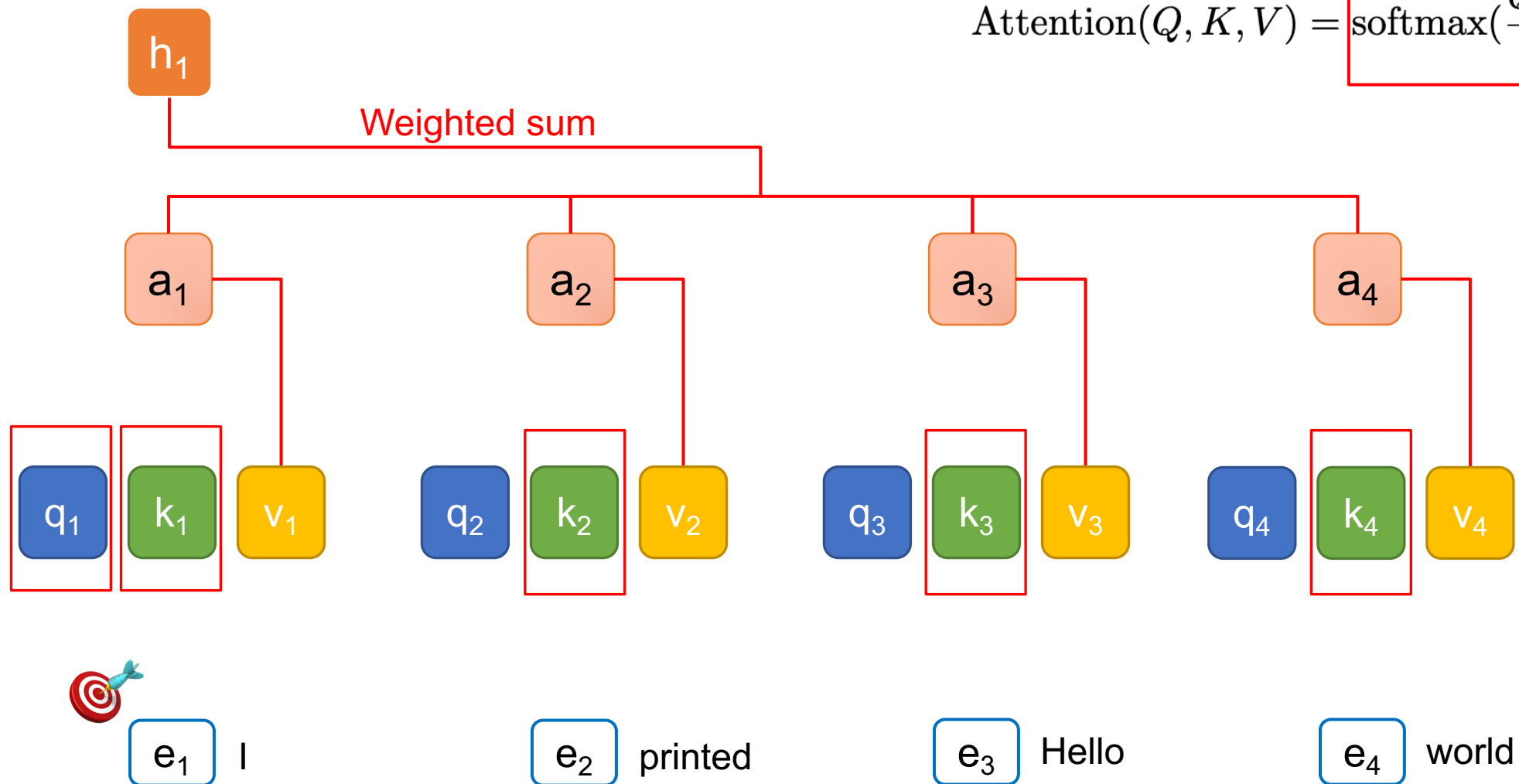


$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Query (Q), Key (K), and Value (V)

t = 1

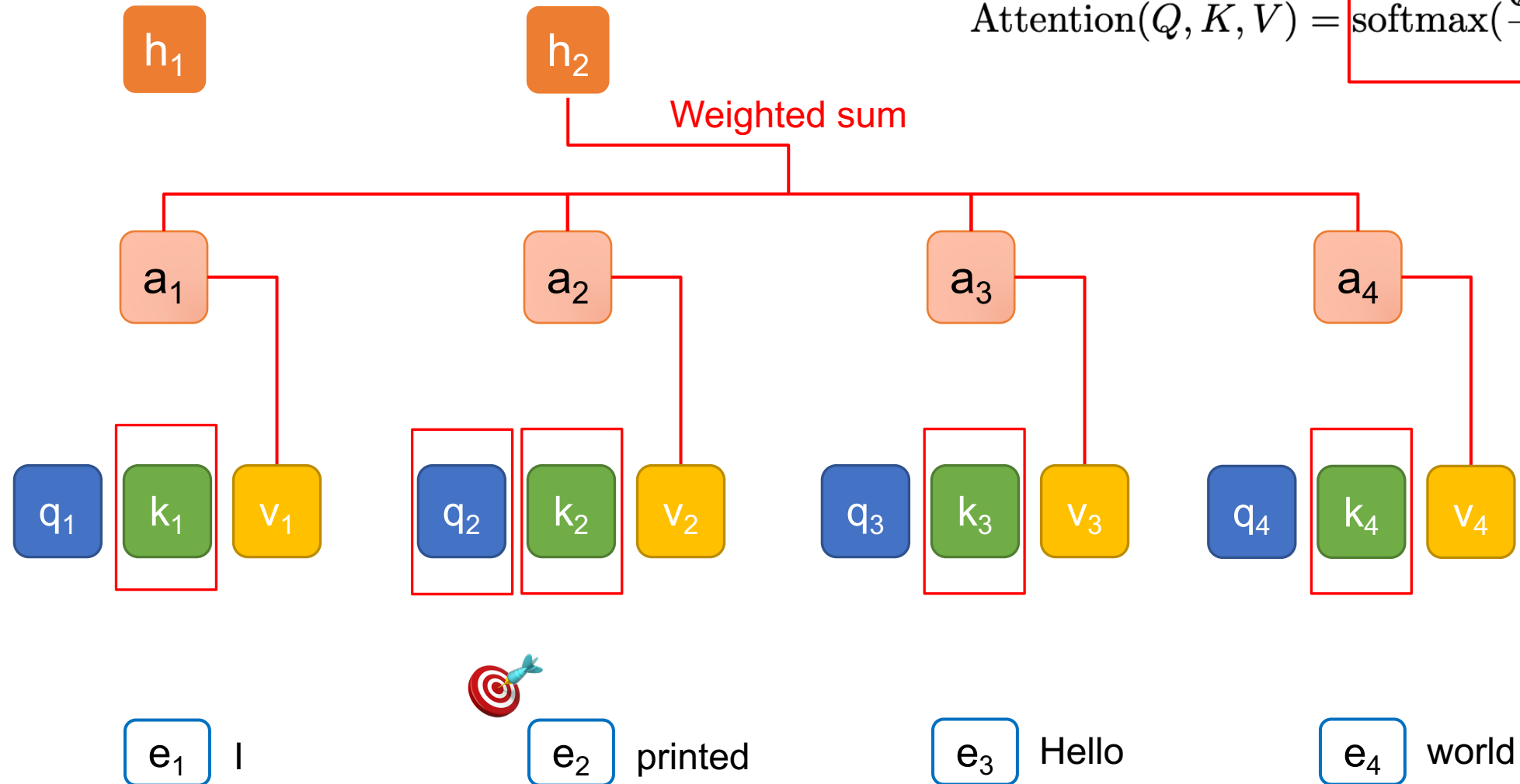
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Query (Q), Key (K), and Value (V)

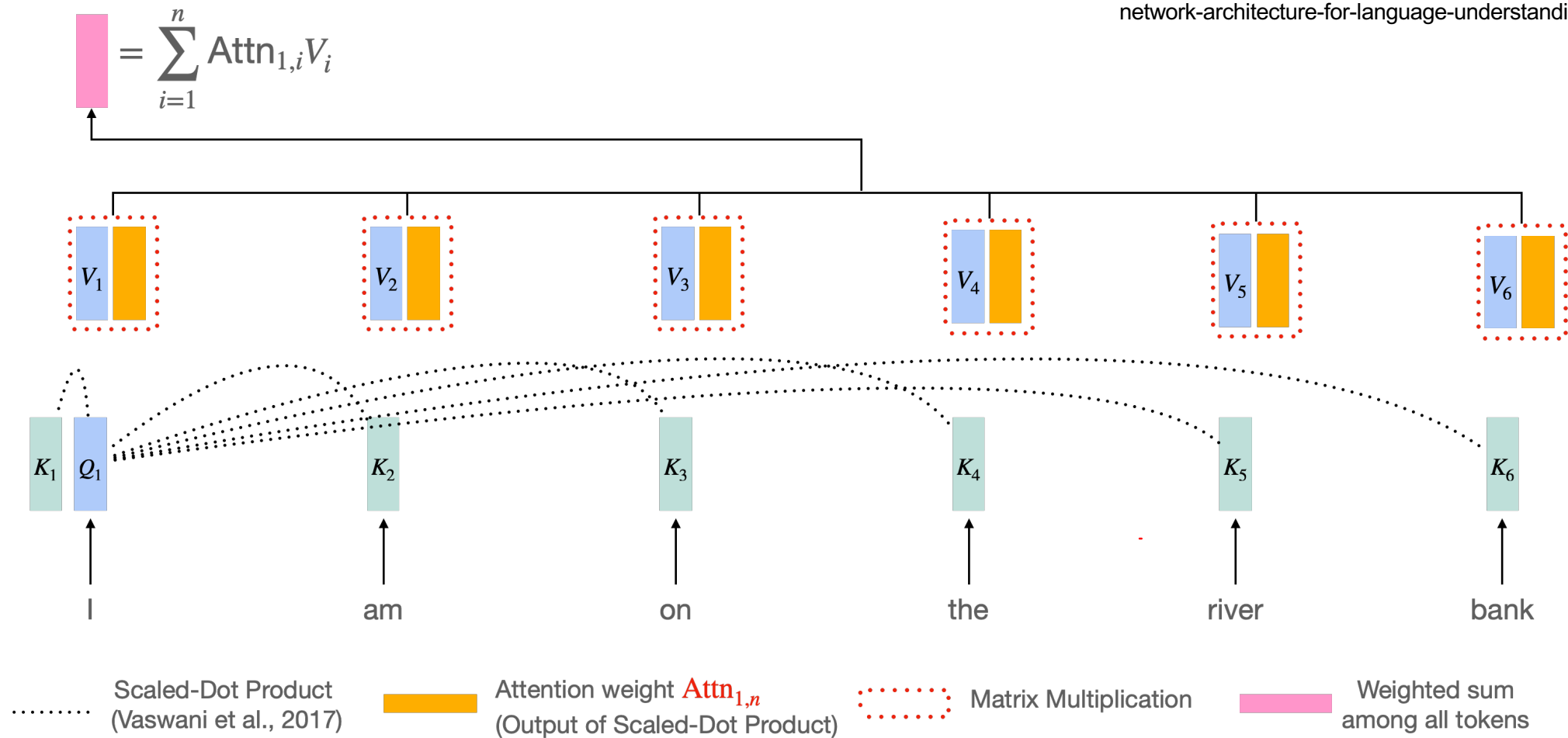
t = 2

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



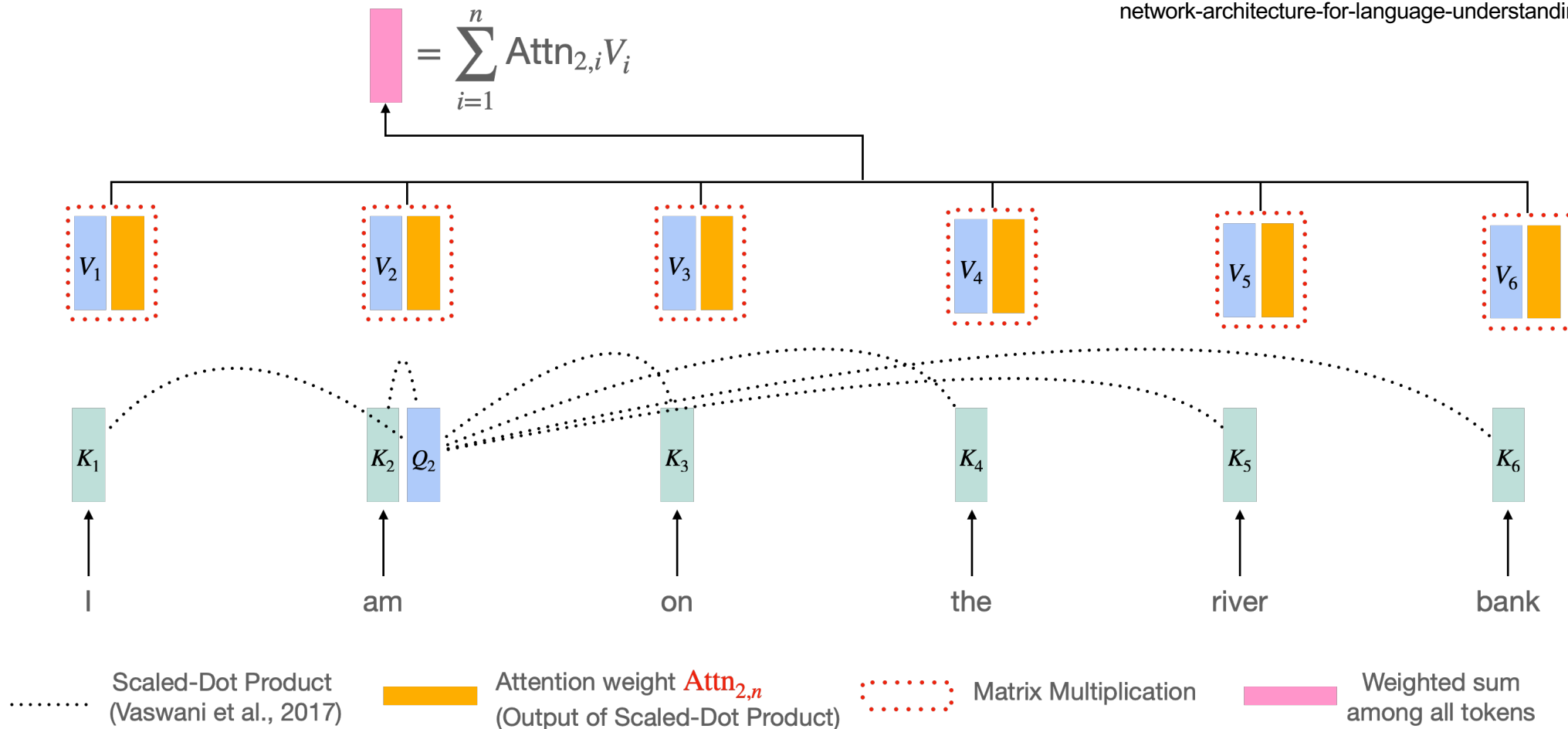
Self-Attention (Transformer) 過程

Vaswani et al. "Attention is all you need." NeurIPS 2017.
<https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/>



Self-Attention (Transformer) 過程

Vaswani et al. "Attention is all you need." NeurIPS 2017.
<https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/>



(For simplicity, I only draw the self-attention process for the second token.)

Self-attention 動畫

Figure source:
<https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/>

Attention Is All You Need

Essential AI

Ashish Vaswani*
Google Brain
~~avaswani@google.com~~

Noam Shazeer*
Google Brain
~~noam@google.com~~

Anthropic
Niki Parmar*
Google Research
~~nikip@google.com~~

Inceptive
Jakob Uszkoreit*
Google Research
~~usz@google.com~~

Cohere

Sakana AI **Llion Jones***
Google Research
~~llion@google.com~~

Aidan N. Gomez* †
University of Toronto
~~aidan@cs.toronto.edu~~

Łukasz Kaiser*
Google Brain
~~lukaszkaizer@google.com~~

OpenAI

NEAR

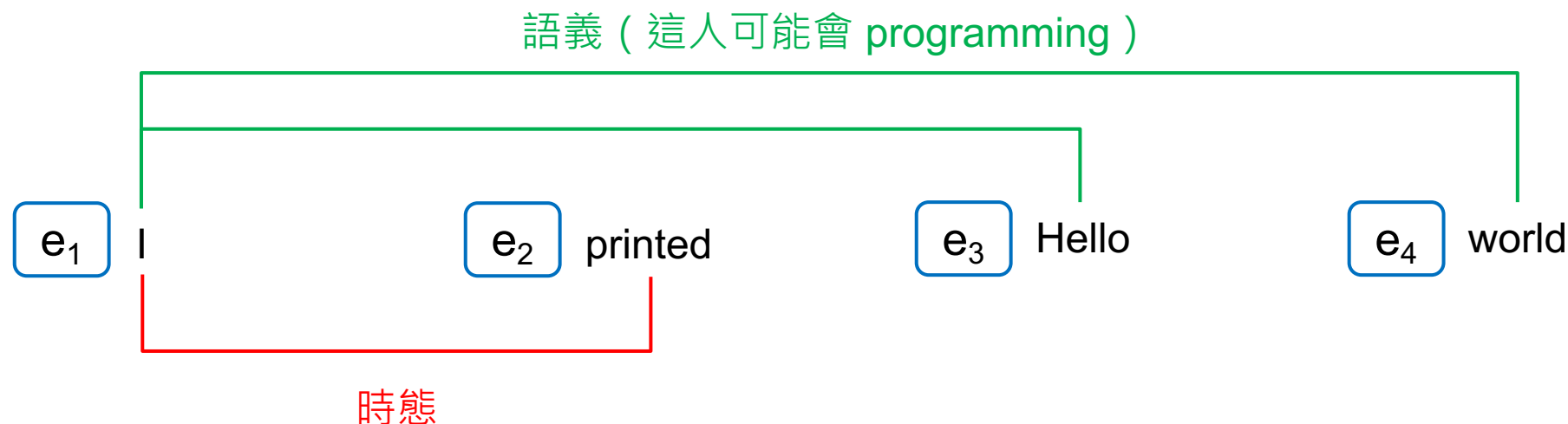
Illia Polosukhin* ‡
~~illia.polosukhin@gmail.com~~

<https://arxiv.org/abs/1706.03762v6>

Multi-Head Attention (MHA)

- 到目前為止，我們已經介紹了 self-attention
 - We **query** the search engine
 - We search engine tries to map our query to the **keys**
 - The outputs are the attended **values** (e.g., the best matched videos).
- 試想：如果我們想要模型注意到多種語言行為呢？

語言存在多種行為 / 模式



h₁

<- Softmax 也是一種 max，只能取得一種最有可能的語言模式

有沒有可能讓模型同時學到多種語言模式？

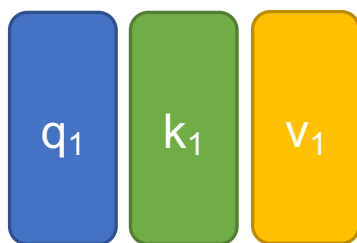
-> Multi-head Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

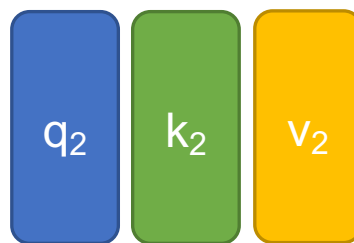
Multi-Head Attention (MHA)

上標: head
下標: token index

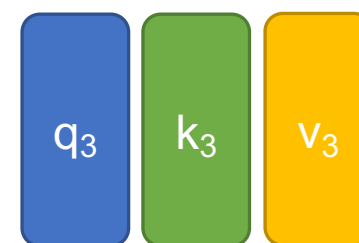
Head = 1 (without MHA)



e_1 I

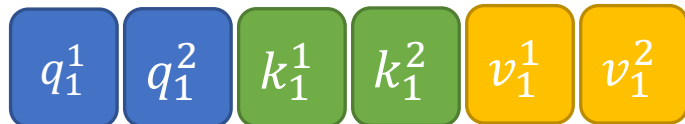


e_2 printed

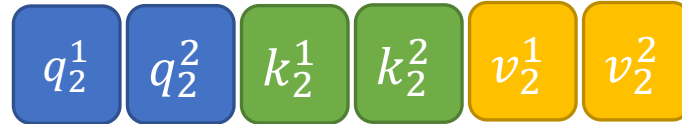


e_3 Hello

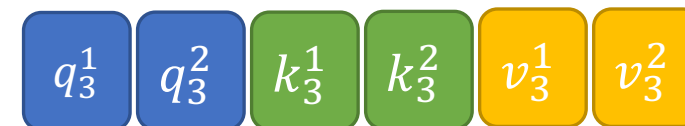
Head = 2 (with MHA)



e_1 I



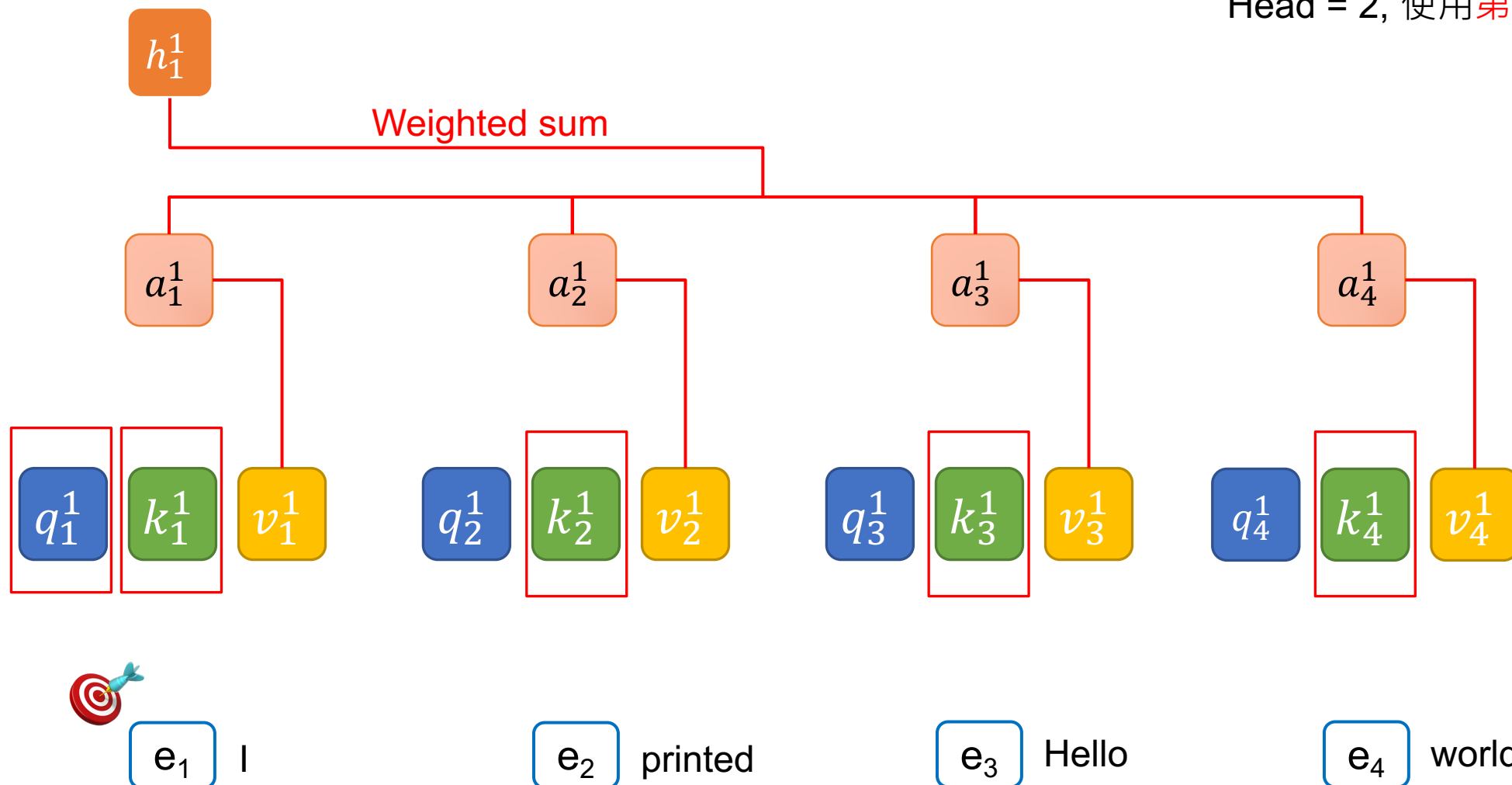
e_2 printed



e_3 Hello

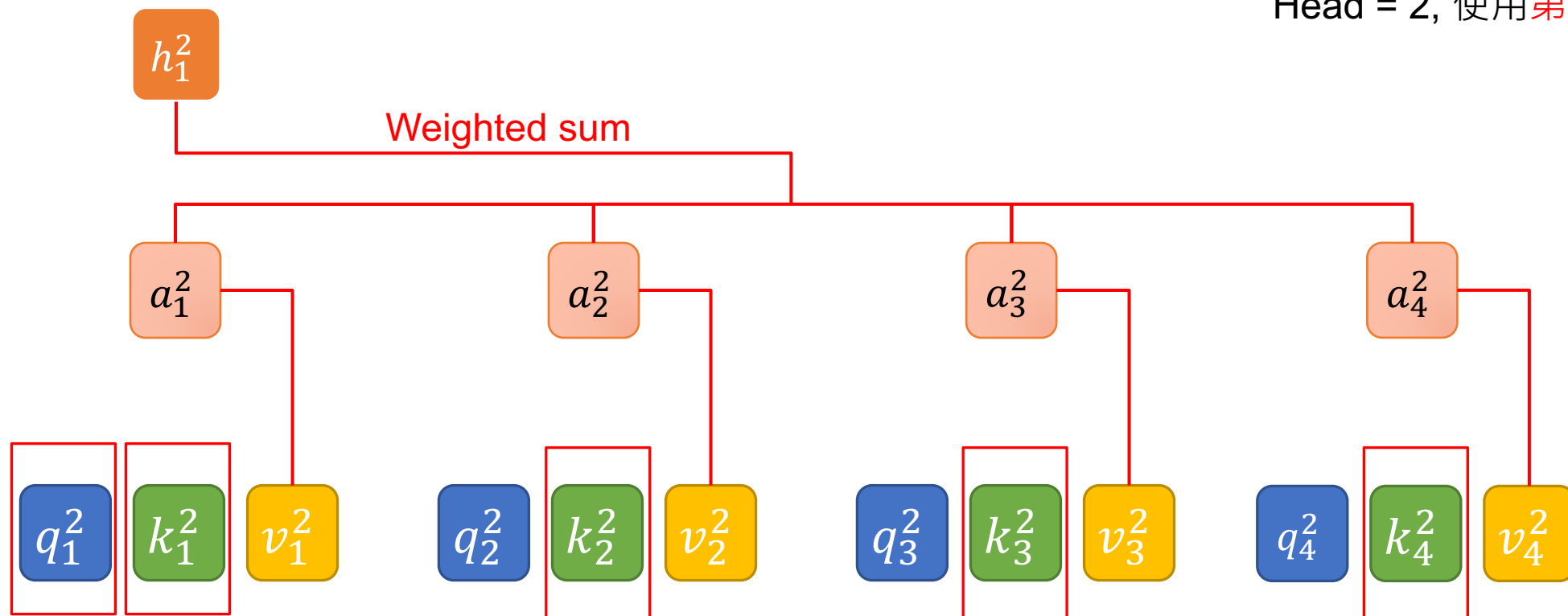
Query (Q), Key (K), and Value (V) with MHA

Head = 2, 使用第一個 head



Query (Q), Key (K), and Value (V) with MHA

Head = 2, 使用第二個 head



e_1 I

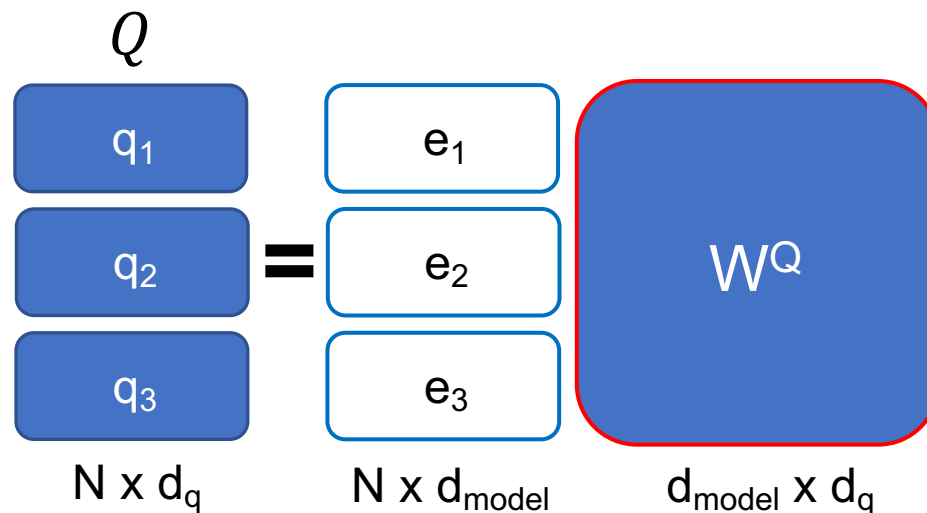
e_2 printed

e_3 Hello

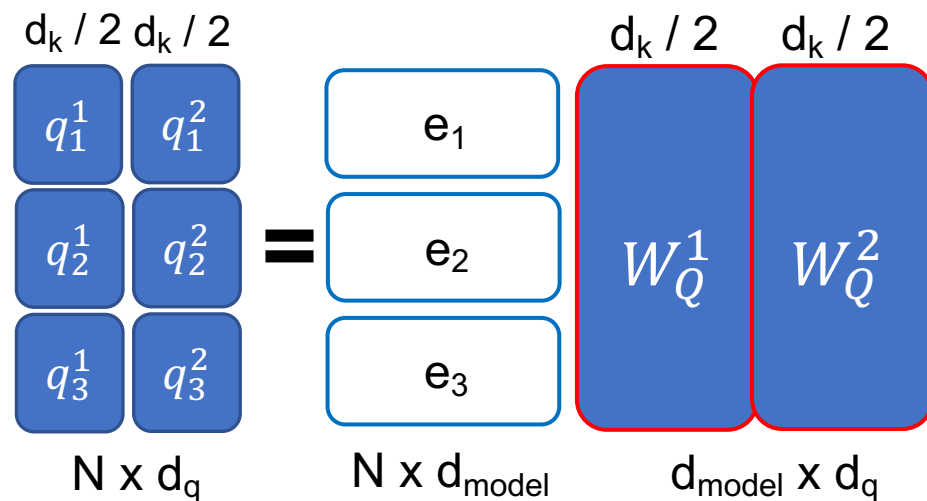
e_4 world

Input Dimensions of MHA

Head = 1 (without MHA)



Head = 2 (with MHA)



Output Dimensions

$$\text{Multihead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

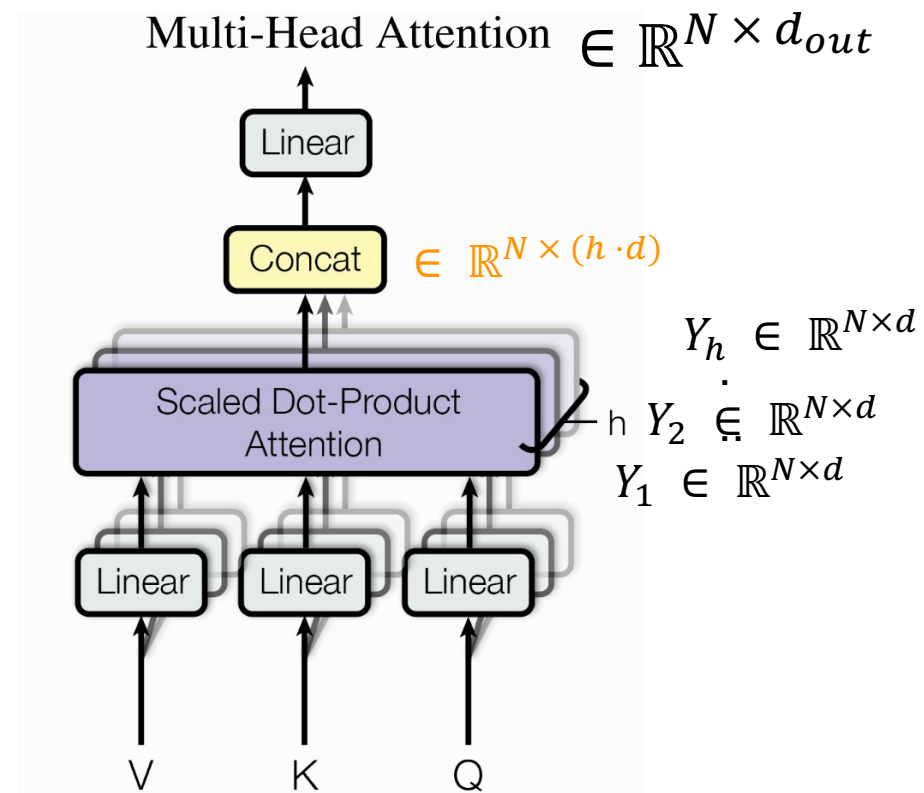
$\in \mathbb{R}^{N \times (h \cdot d)}$
 $\in \mathbb{R}^{(h \cdot d) \times d_{out}}$

Head = 1 (without MHA)

$$h_1 \times W^O = h_1$$

Head = 2 (with MHA)

$$\begin{bmatrix} h_1^1 \\ h_1^2 \end{bmatrix} \times W^O = h_1$$



MHA 比較好嗎？

Clark, Kevin, et al. "What Does BERT Look at? An Analysis of BERT's Attention." Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP. 2019.

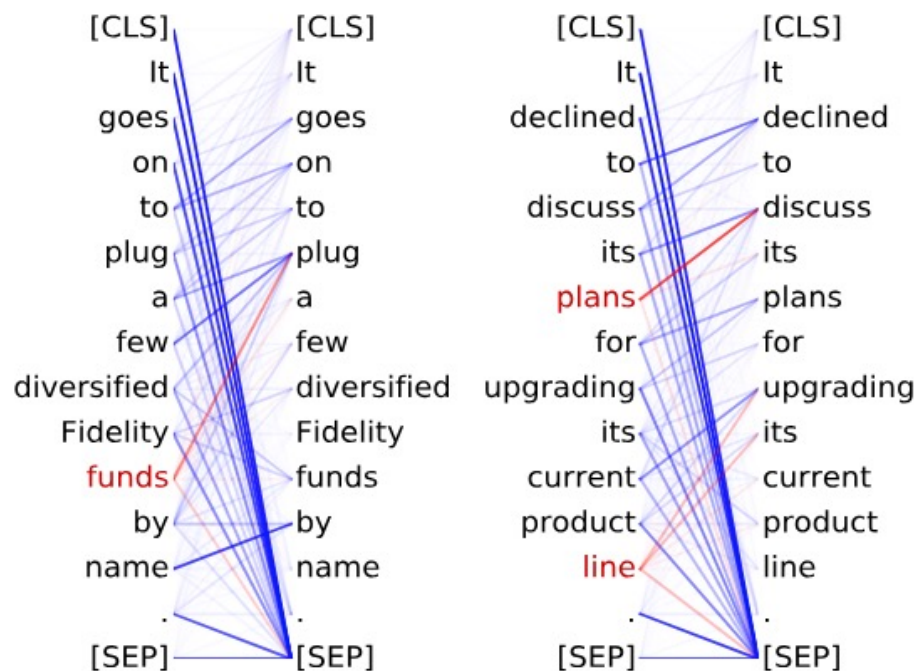
實驗證明：不同 head 關注的地方不太一樣

<layer>-<head number>

Head 8-10

關注受詞

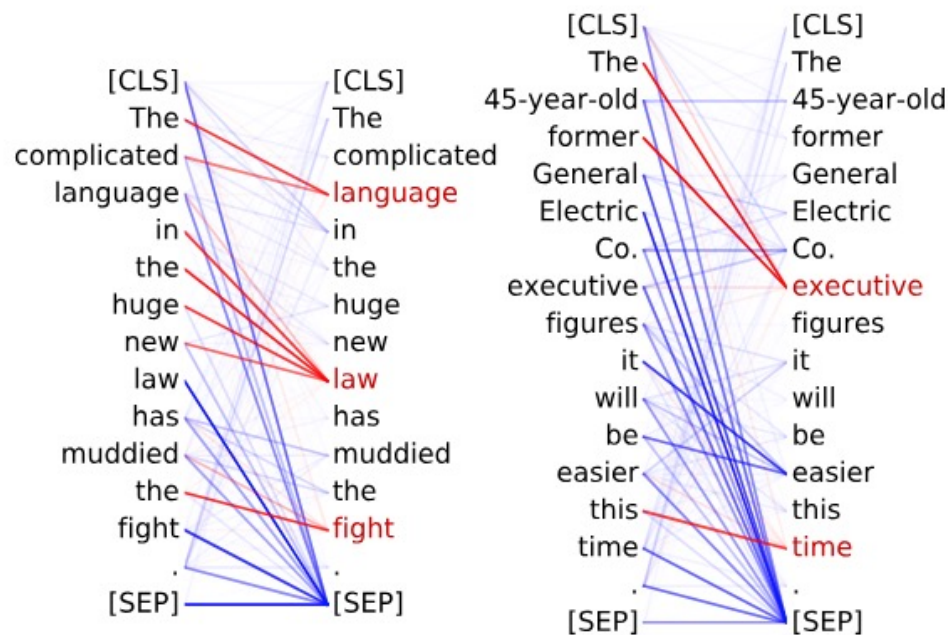
- **Direct objects** attend to their verbs
- 86.8% accuracy at the **dobj** relation



Head 8-11

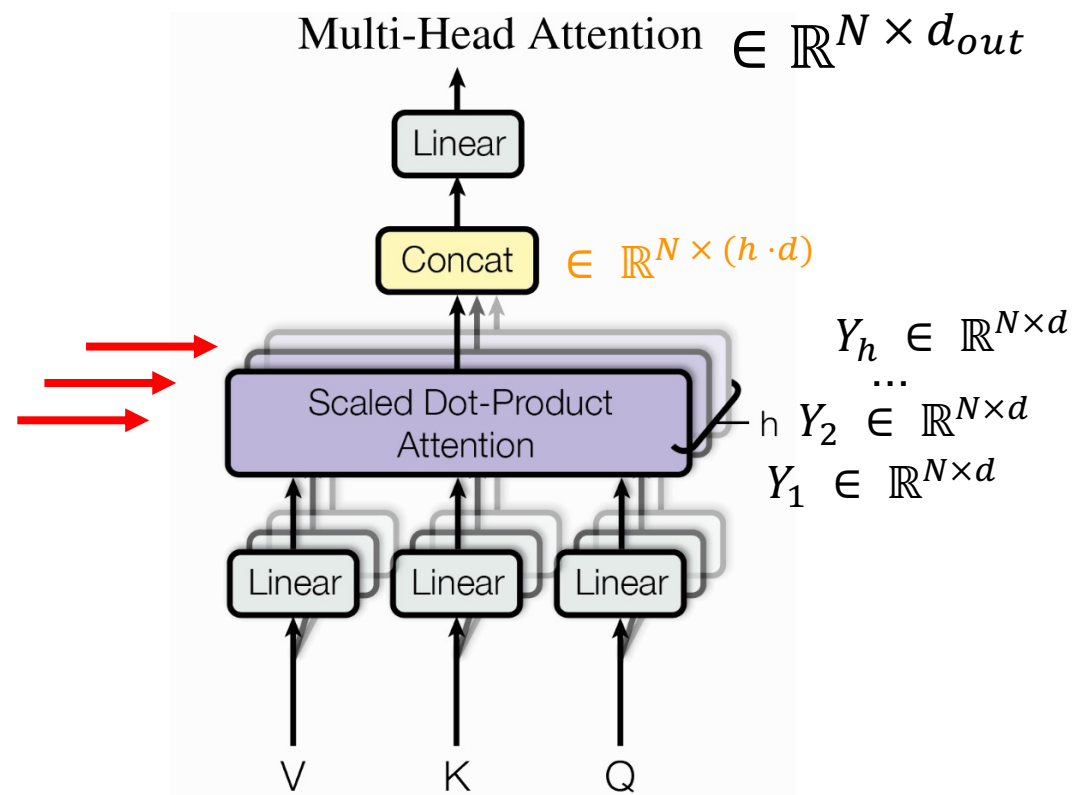
關注限定詞

- **Noun modifiers** (e.g., determiners) attend to their noun
- 94.3% accuracy at the **det** relation



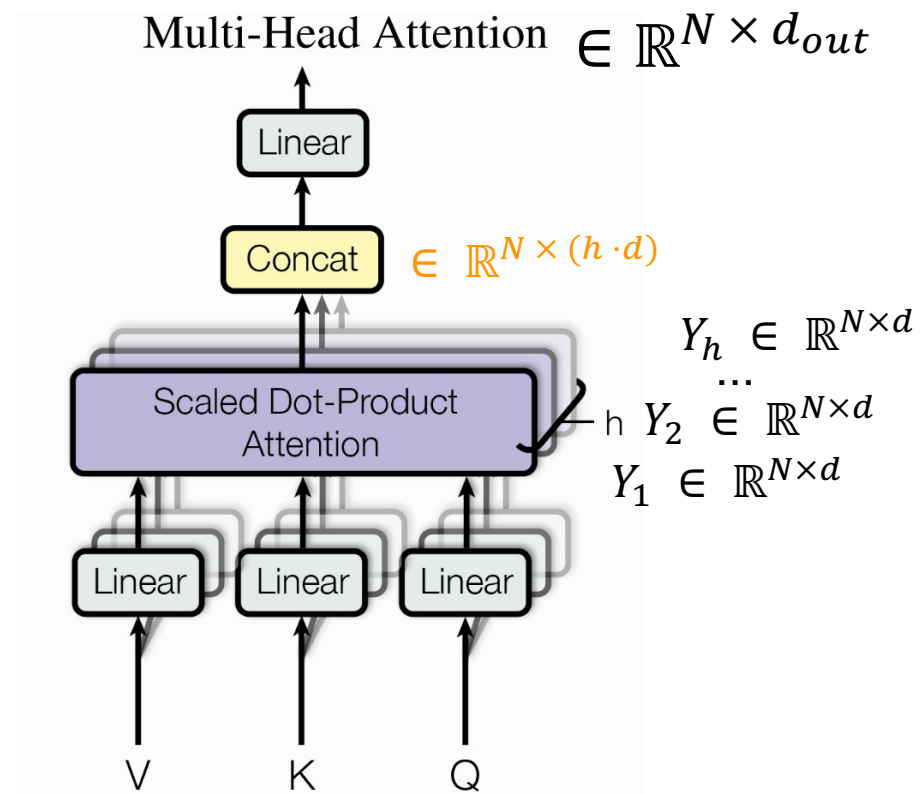
MHA 的過程中涵蓋了多次 softmax

每次 scaled dot-product
attention 都會做 softmax



Summary of MHA

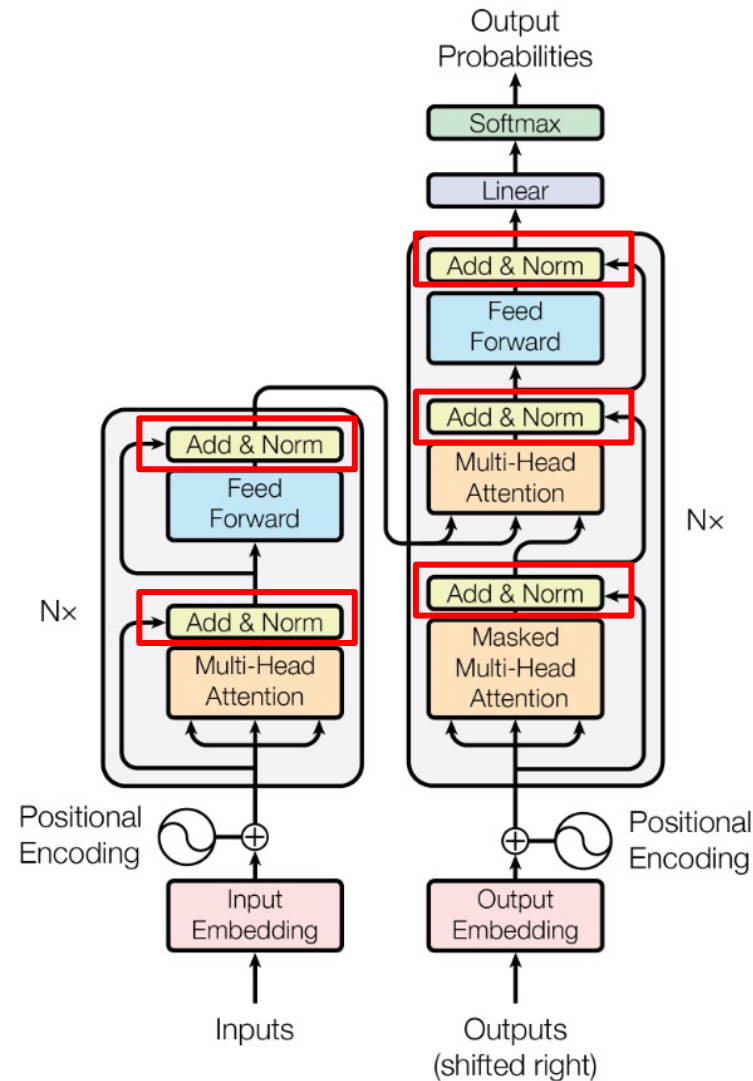
- 透過 MHA，模型會將表示投影到多個不同的子空間，讓每個 head 各自進行 self-attention，因此同一個時間點可以同時從不同角度捕捉序列中的關係
- 標準 Transformer 的 num_heads 通常不改變最終輸出維度，因為總模型維度 d_model 固定，head 數改變時會相應調整每個 head 的維度



Outline of Transformer

- Add: Residual Connection
- Norm: Layer Normalization

Vaswani, Ashish, et al. "Attention is all you need." NeurIPS (2017).



Add & Norm

- Research shows that Residual Connection (He, Kaiming, et al., 2016) stabilizes training.
- Let the layers in between learn the residual relationship (i.e. $Y - X$).

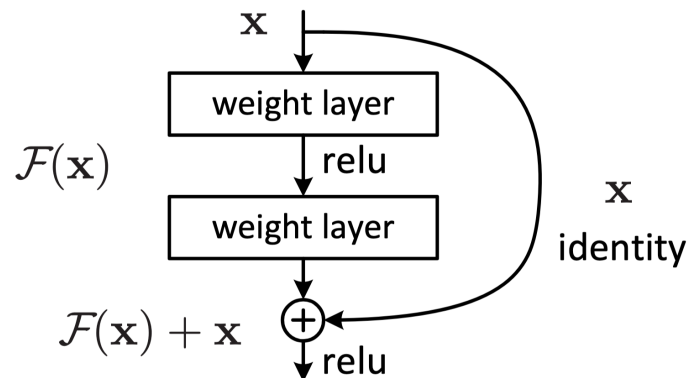
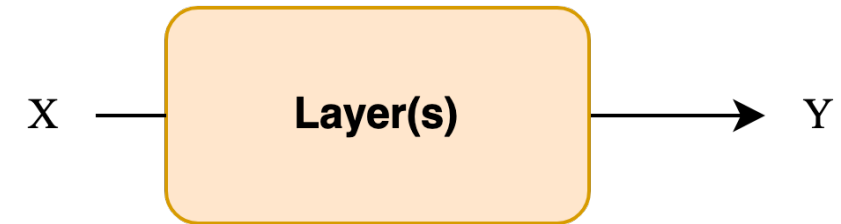
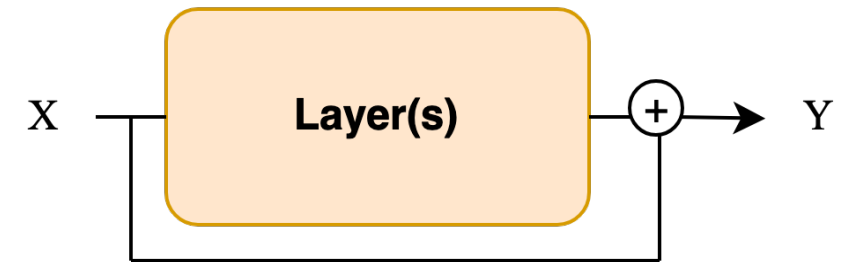


Figure 2. Residual learning: a building block.

Standard



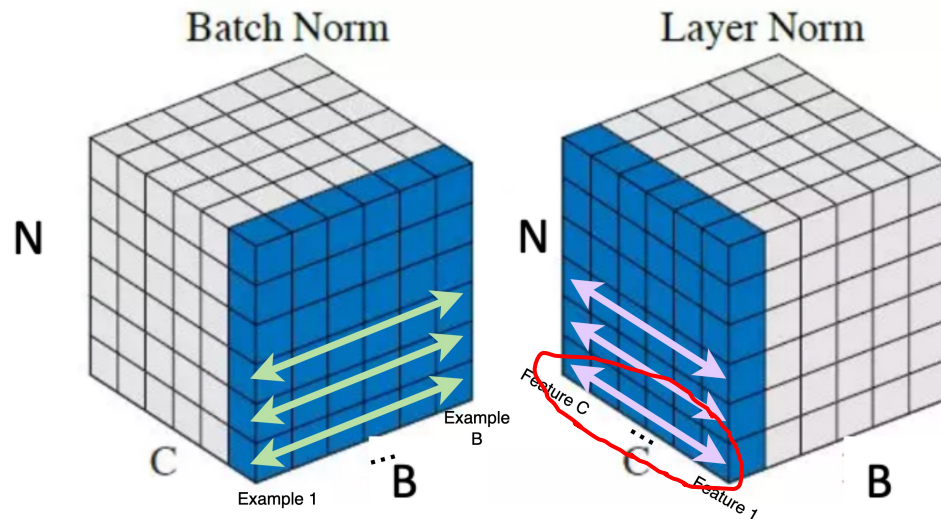
Residual



Add & Norm (Layer Normalization)

■ Layer Norm: For an input vector, calculate its mean and std across embedding dimension (within an example).

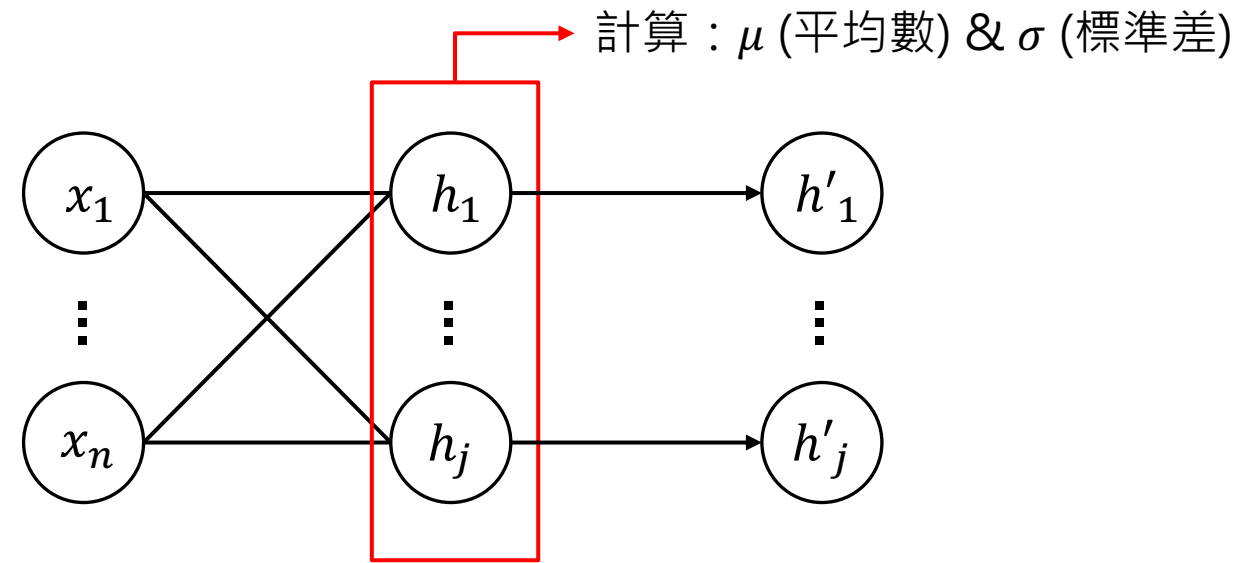
■ $\text{norm}(X) = \frac{X - \mu}{\sigma}$



B: Batch Size

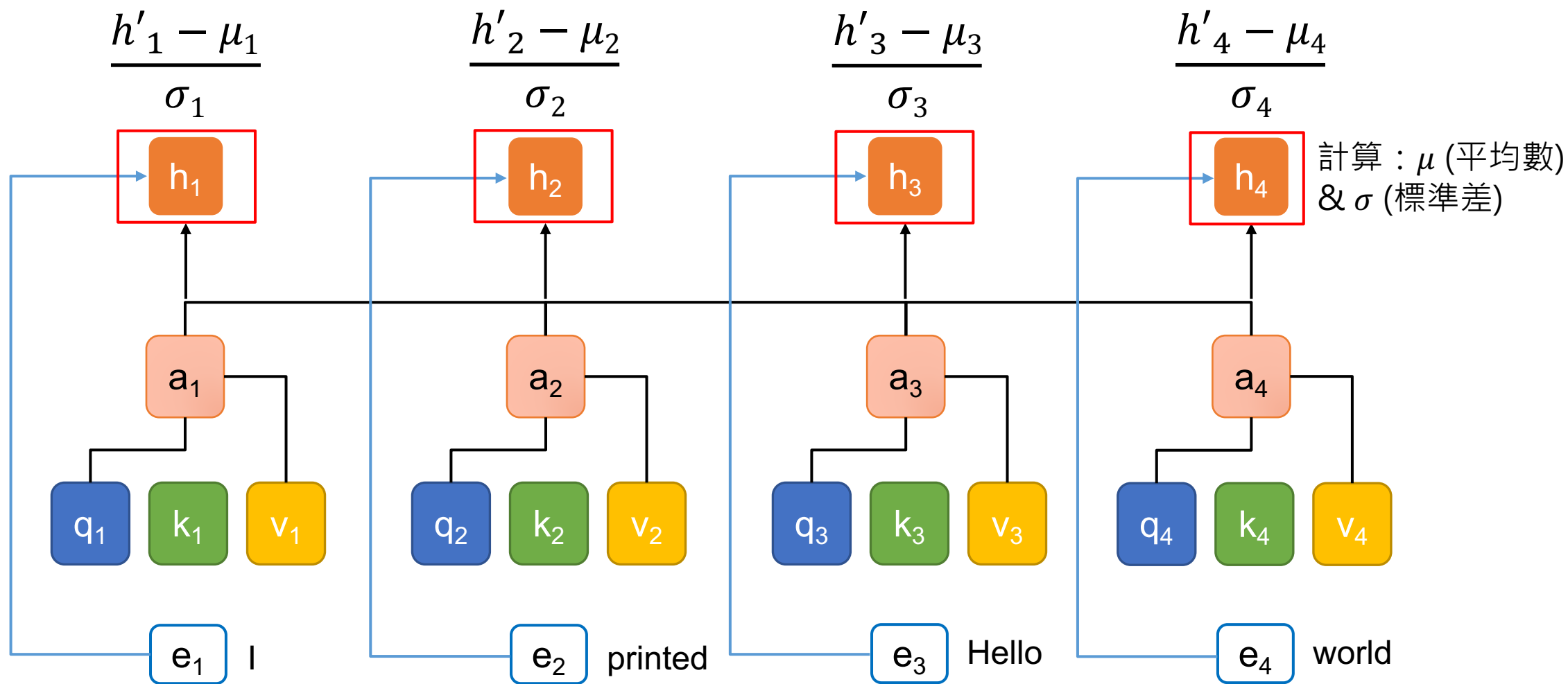
C: Channels / Feature Dimension

N: Sequence length (CNN 中代表長x寬的數目)



(Batch Normalization)

Add & Norm (Layer Normalization)



Why LN not BN?

- 資料長度差距
- 模型實作表現差異

[Intro] Static padding

batch 1

I	love	CGU	[PAD]	[PAD]	[PAD]
Yesterday	I	watched	a	good	movie
Your	new	shoes	look	nice	[PAD]

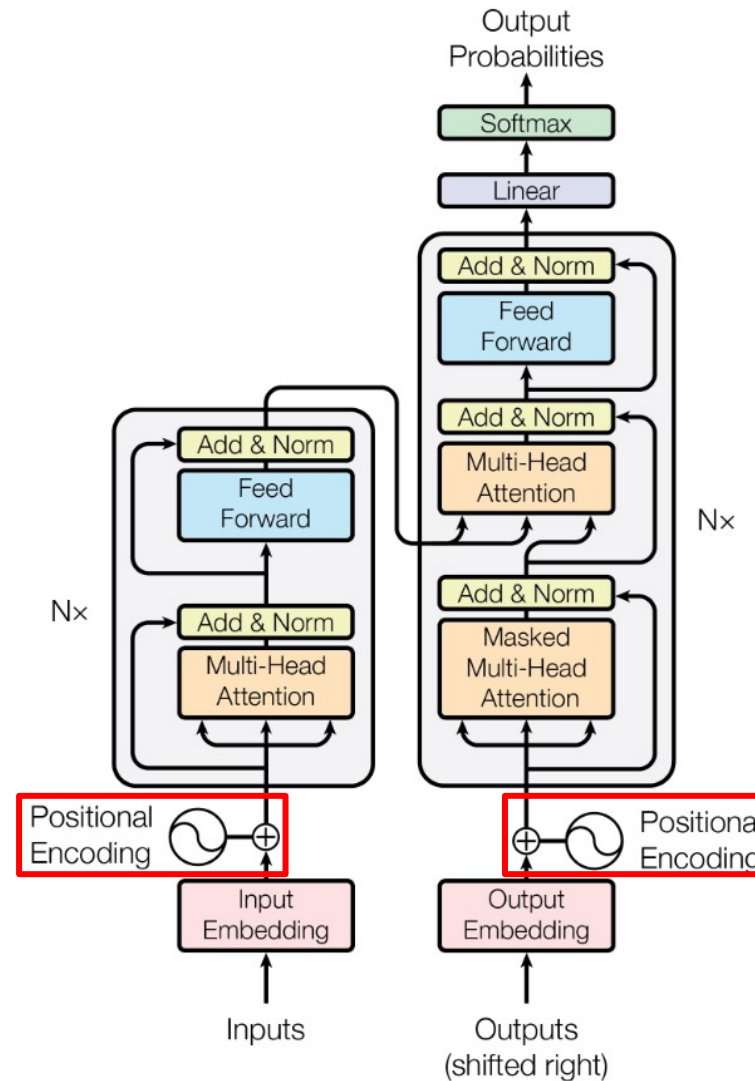
longest

batch 2

Merry	Christmas	[PAD]	[PAD]	[PAD]	[PAD]
I	love	coding	[PAD]	[PAD]	[PAD]
Me	[PAD]	[PAD]	[PAD]	[PAD]	[PAD]

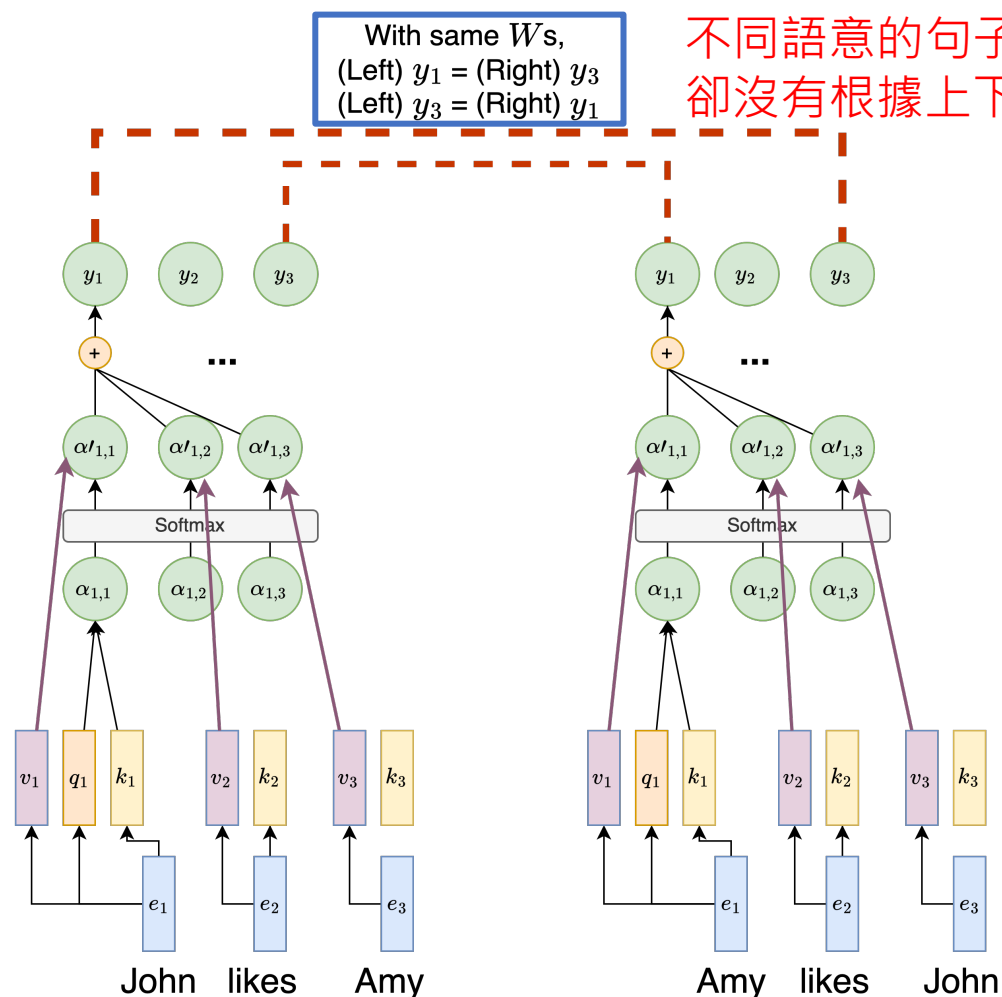
Outline of Transformer

Vaswani, Ashish, et al. "Attention is all you need." NeurIPS (2017).

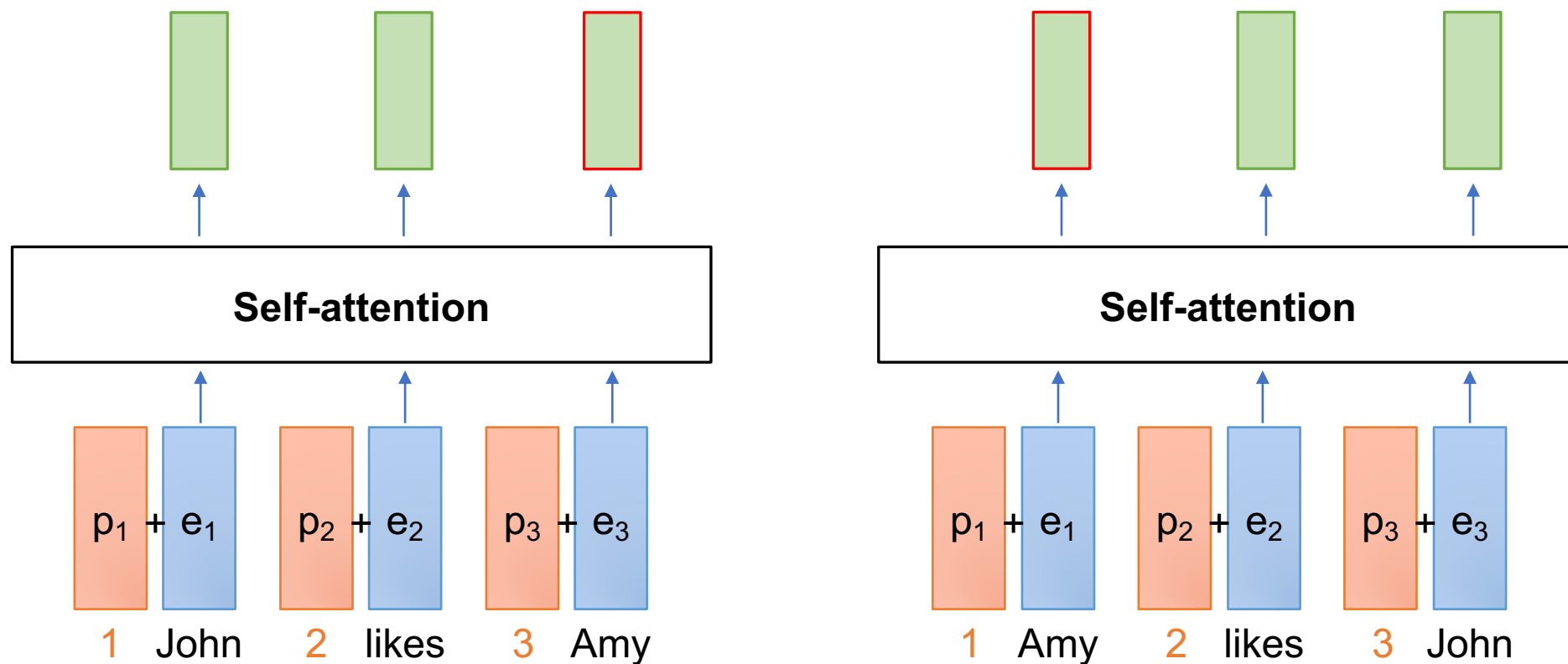


Lack of Position Modeling

- RNN 有位置資訊 (因為 RNN step-by-step 處理文字)
- 然而，Transformer 中 self-attention 的操作遺失了 step-by-step 處理文字的方式，故缺少位置資訊



常見位置資訊作法：Learned Positional Embeddings



Learned Positional Embeddings 程式碼

https://huggingface.co/transformers/v2.1.1/_modules/transformers/modeling_bert.html

```
class BertEmbeddings(nn.Module):
    """Construct the embeddings from word, position and token_type embeddings.
    """
    def __init__(self, config):
        super(BertEmbeddings, self).__init__()
        self.word_embeddings = nn.Embedding(config.vocab_size, config.hidden_size, padding_idx=0)
        self.position_embeddings = nn.Embedding(config.max_position_embeddings, config.hidden_size)
        self.token_type_embeddings = nn.Embedding(config.type_vocab_size, config.hidden_size)

        # self.LayerNorm is not snake-cased to stick with TensorFlow model variable name and be able to load
        # any TensorFlow checkpoint file
        self.LayerNorm = BertLayerNorm(config.hidden_size, eps=config.layer_norm_eps)
        self.dropout = nn.Dropout(config.hidden_dropout_prob)
```

缺點：Learned Positional Embeddings 不會超出 max_position 之後的資訊

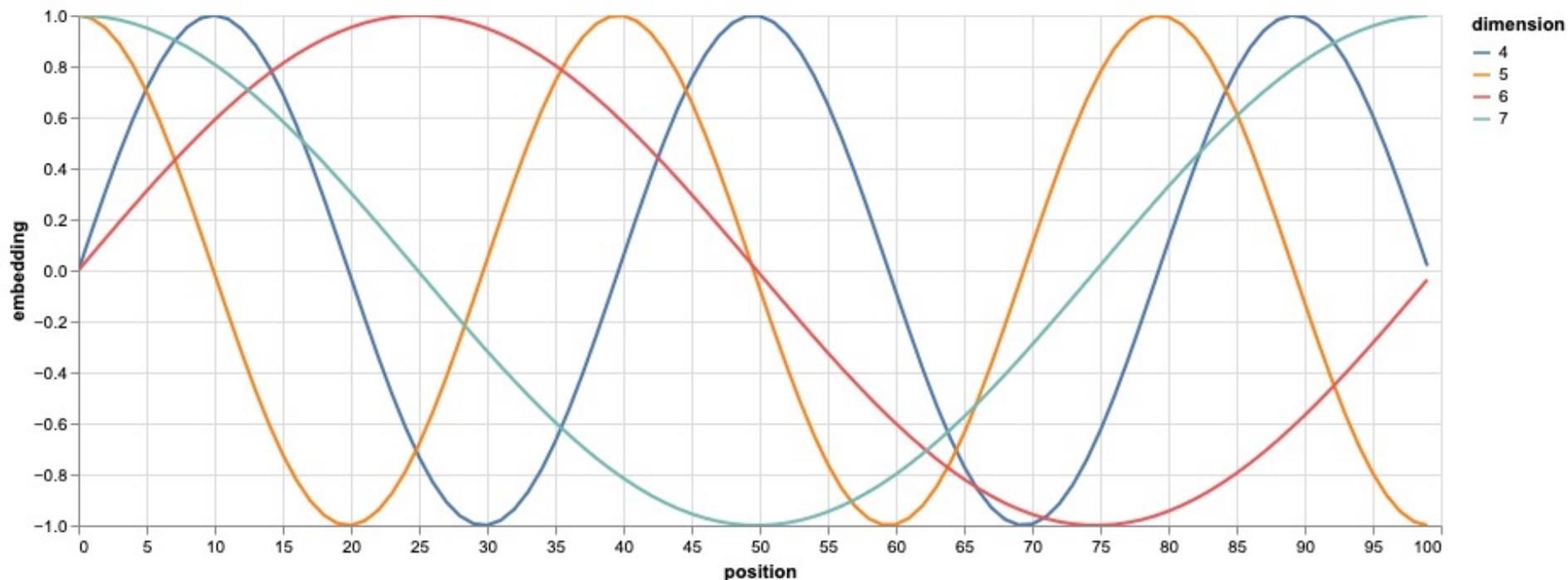
Positional Encoding in Transformer

- 創造出相對位置的 embeddings
- 核心精神：利用三角函數的週期性來建構序列資訊
 - 偶數的 embedding 單元 ($2i$, $i = 0, 2, 4, \dots, d_{\text{model}} / 2 - 2$) : Sin 函數
 - 奇數的 embedding 單元 ($2i+1$, $i = 1, 3, 5, \dots, d_{\text{model}} / 2 - 1$) : Cosine 函數

$$PE_{(pos, 2i)} = \sin(\underbrace{pos}_{\text{輸入序列中任一token的位置}} / \underbrace{10000^{2i/d_{\text{model}}}}_{\text{i越大 -> 分母越大 -> 相位變化越慢 -> 頻率越小}})$$
$$PE_{(pos, 2i+1)} = \cos(\underbrace{pos}_{\text{輸入序列中任一token的位置}} / \underbrace{10000^{2i/d_{\text{model}}}}_{\text{10000是實驗得出的較佳數值}})$$

輸入序列中任一token的位置

Positional Encoding Visualization



- Question: 為什麼需要同時有 Sin 和 Cosine 的函數？
- Ans: 藉由不同週期性來捕捉序列中各個位置的關係

$$\sin 30^\circ = 0.5$$

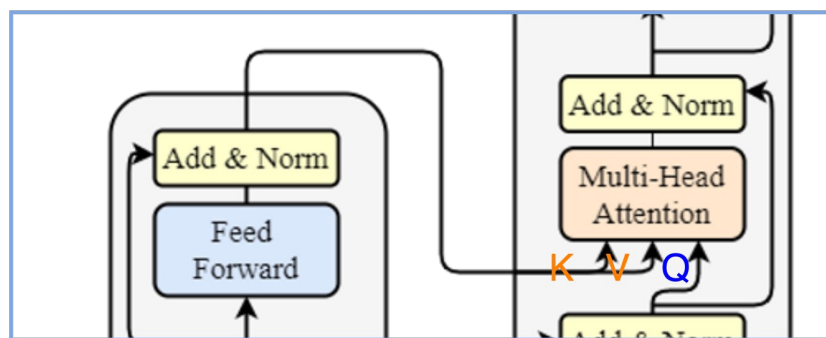
$$\sin 150^\circ = 0.5$$

vs.

$$(\sin 30^\circ, \cos 30^\circ) = (0.5, 0.866)$$

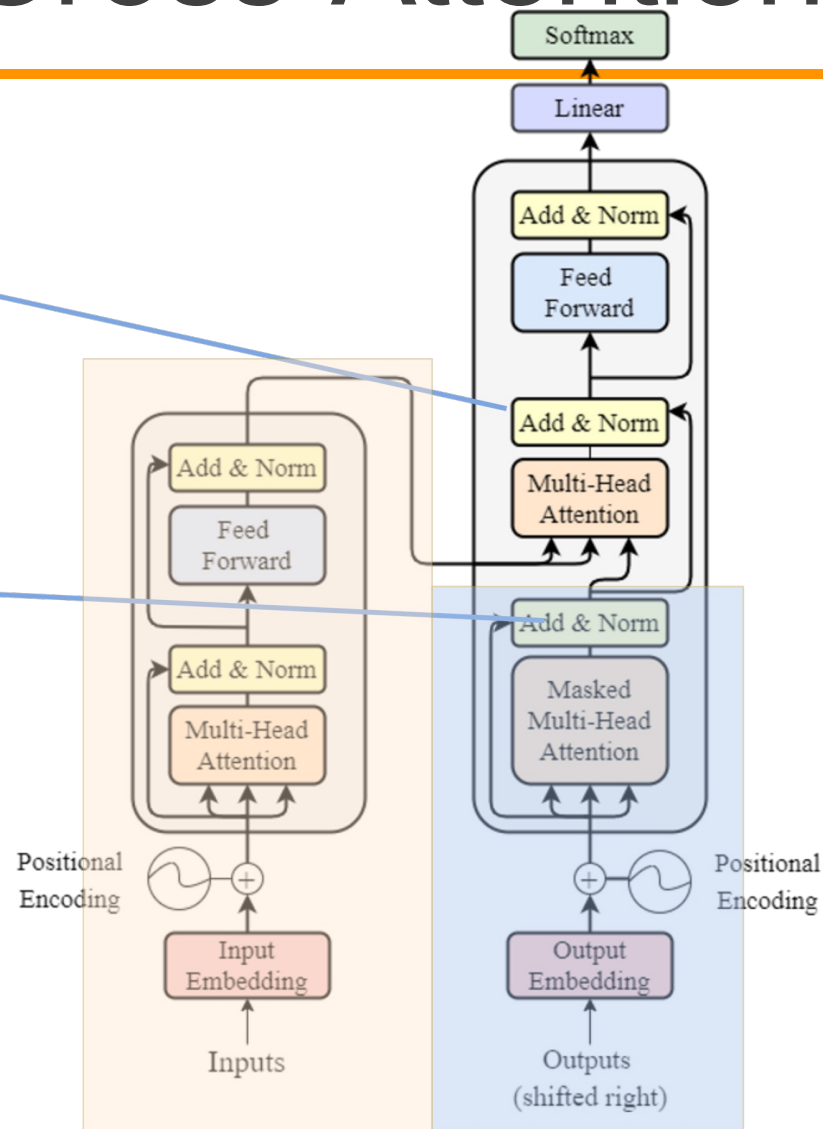
$$(\sin 150^\circ, \cos 150^\circ) = (0.5, -0.866)$$

Encoder-Decoder 橋樑: Cross-Attention



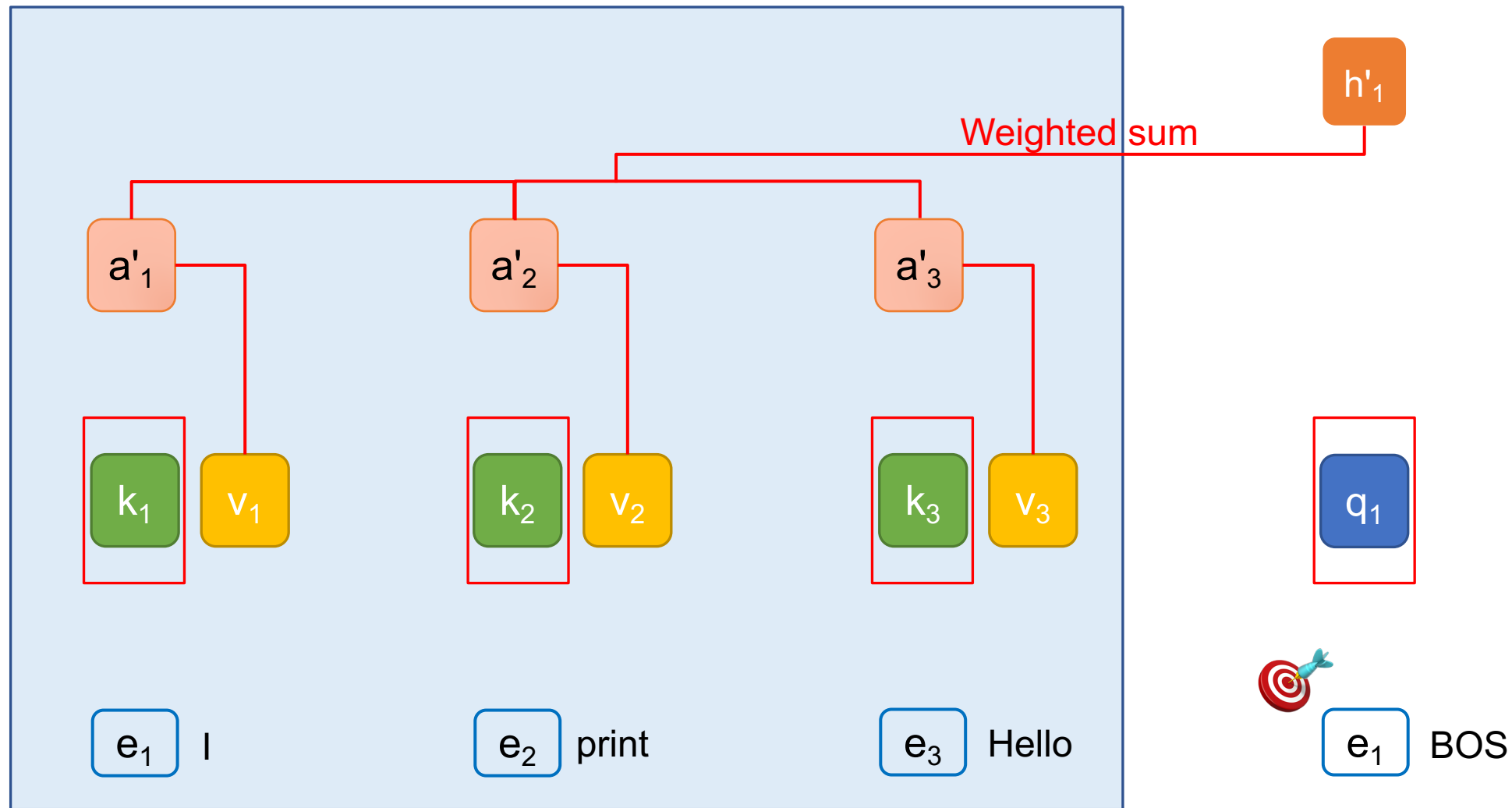
K, V uses the output from encoder as their X;
Q uses the decoder's information as X.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Cross-attention

Encoder



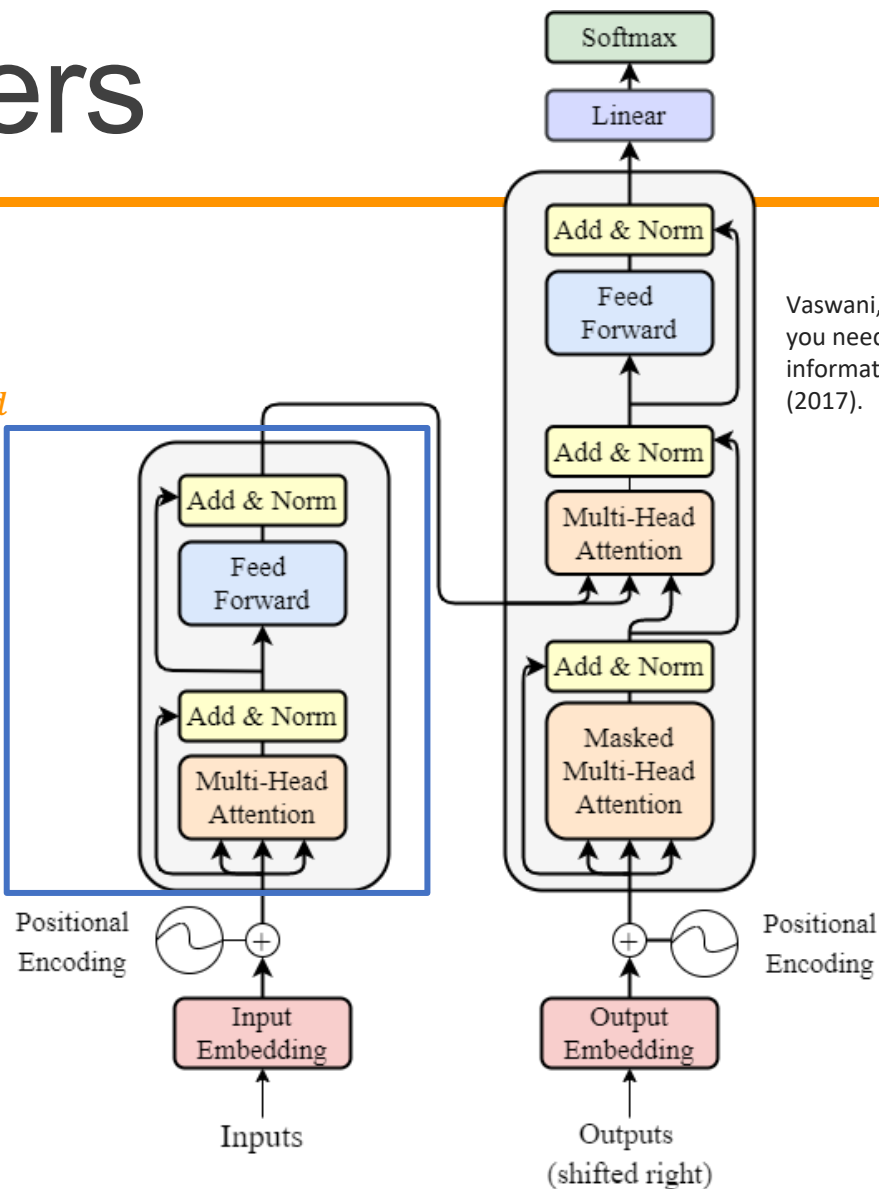
Number of Layers

Input, output dimension match!
We can have multiple encoders stacked up, to increase the number of trainable parameters.

$$\text{output} \in \mathbb{R}^{N \times d}$$

N *

$$\text{input} \in \mathbb{R}^{N \times d}$$



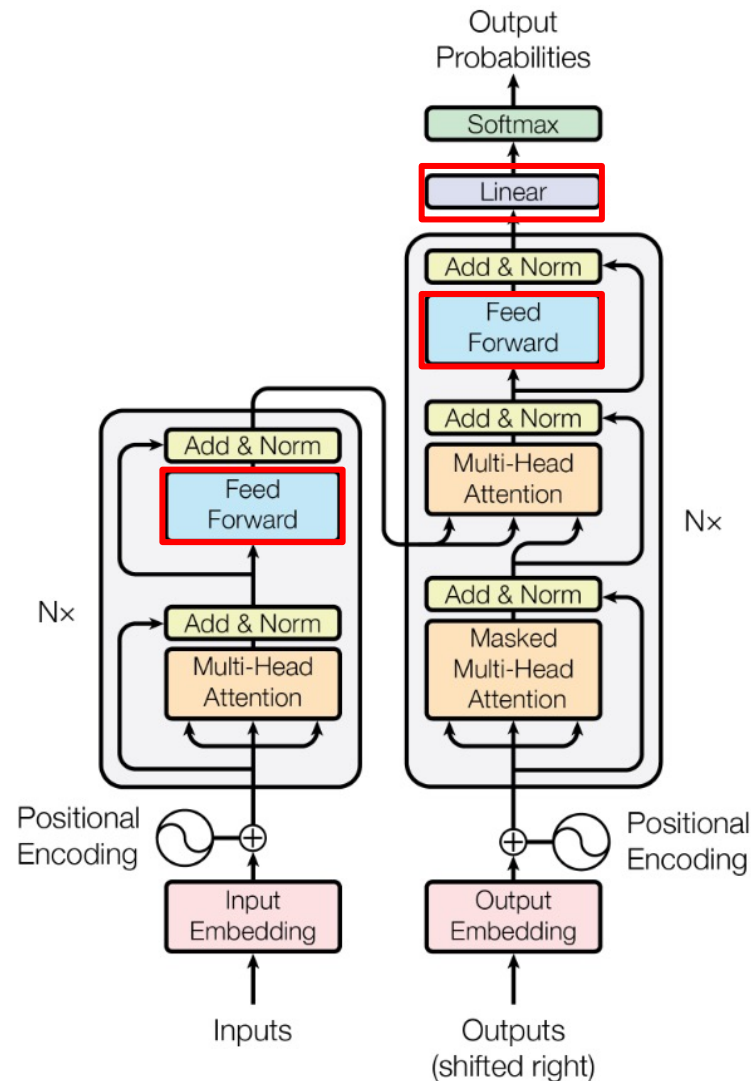
Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).

Outline of Transformer

Feed Forward Networks:

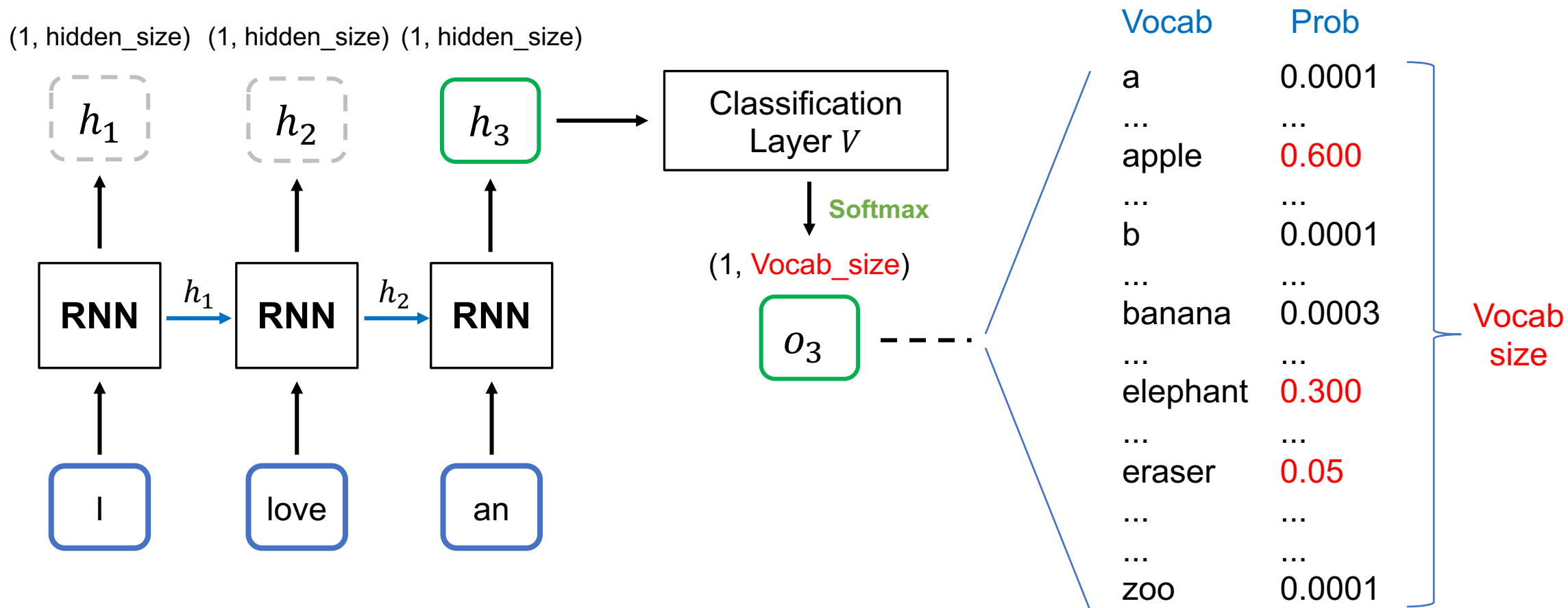
- Simply multiply by a weight matrix (MLP).
- Used to project to a specific dimension.

Vaswani, Ashish, et al. "Attention is all you need." NeurIPS (2017).



[Recap] Text Generation with RNN

*本頁的RNN不包含輸出層 V



Transformers' Achievements

1. In original paper, State-of-The-Art (SoTA) of machine translation.

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

6 encoders,
6 decoders,
h = 512, 100K train
steps.

6 encoders,
6 decoders,
h = 1024, 300K
train steps.

Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).
<https://arxiv.org/abs/1706.03762>

Transformer Variants (there are many more)

- Universal Transformer

- Adds Adaptive Computation Time
- Dehghani, Mostafa, et al. "Universal transformers." arXiv preprint arXiv:1807.03819 (2018).

- Longformer

- Tackles long documents
- Iz Beltagy, et al. (2020). Longformer: The Long-Document Transformer. arXiv:2004.05150.

- Roformer

- Changes the design of positional encoding
- Su, Jianlin, et al. "RoFormer: Enhanced Transformer with Rotary Position Embedding. CoRR abs/2104.09864 (2021)." arXiv preprint arXiv:2104.09864 (2021).

- Vision Transformer

- Tackles vision tasks
- Alexey Dosovitskiy, et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale." International Conference on Learning Representations. 2021.

Additional Links

- <https://nlp.seas.harvard.edu/annotated-transformer/>
- <https://datascience.stackexchange.com/questions/82451/why-is-10000-used-as-the-denominator-in-positional-encodings-in-the-transformer>

Thank you!

Instructor: 林英嘉

 yjlin@cgu.edu.tw

TA: 林宣辰

 b1128015@cgu.edu.tw